

The Wang Method Series
Notebooks #1–#9 (Version 1.0)
Understanding Larger Rubik's Cubes
A Beginner's Guide

Laung-Terng (LT) Wang
September 9, 2026

Acknowledgments

I would like to express my sincere appreciation to the Rubik's Cube community for generously sharing educational videos, tutorials, solving techniques, and ideas through public websites and online communities.

I am especially grateful to the creators of outstanding educational resources, including Claudia Carlucci, CubeSkills by Feliks Zemdeg, J Perm, CubeHead, Rubik'SpeedyGoneCuber by Raphael "Rafi" Niman, EasiestSolve, Noah Richardson, and Rubik'Snap. Their tutorials and explanations enriched my understanding and inspired me to explore simpler approaches.

I would also like to thank Marty Kelsey (Martyt7777), bcube, and Lee Dearn (stoic) of the SpeedSolving.com community for their suggestions and willingness to share their knowledge.

The observations, experiments, classifications, and methods presented in *The Wang Method Series* were independently developed through my own analysis and engineering perspective, with inspiration from the supportive learning environment created by the worldwide cubing community.

I hope these notebooks will encourage beginners to discover that solving larger Rubik's Cubes is not only about learning algorithms, but also about observation, reasoning, experimentation, and the joy of sharing knowledge.

Additional materials and updates for *The Wang Method Series* are available on the author's website: <https://ltwang888.github.io/>

Table of Contents

Notebook #1

Understanding the Top-Layer Four-Edge-Pair Patterns 3

Notebook #2

Understanding the Last Three Edge Pairs in the Yau 3-2-3 Pairing Strategy 8

Notebook #3

Structural Classification of the Last Three Edge Pairs 12

Notebook #4

Wang's Simple L3E Pairing Method 23

Notebook #5

Structural Classification and State Transition of the 24 Top-Layer Four-Edge-Pair Patterns 34

Notebook #6

Understanding the Last Two Centers of the 5×5 Rubik's Cube 39

Notebook #7

Comparative Study and Educational Perspectives 51

Notebook #8

A Minimal-Memory Approach to 6×6 Last Two Centers (L2C) 58

Notebook #9

Understanding Edge Parity on Odd-Order Rubik's Cubes 63

The Wang Method Series

Notebook #1 (Version 1.0)

Understanding the Top-Layer Four-Edge-Pair Patterns

Unusual Top-Layer Four-Edge-Pair Patterns After OLL Parity Correction

Laung-Terng (LT) Wang

August 1, 2026

1. Motivation

While solving several 4×4 Rubik's Cubes using the CFOP Method followed by reduction, I repeatedly observed unusual top-layer four-edge-pair patterns after correcting the OLL parity and completing the remaining standard 3×3 OLL algorithms.

Instead of encountering only the familiar PLL parity cases, the cube occasionally reached several apparently abnormal four-edge-pair configurations. These patterns did not resemble the standard opposite-edge PLL parity cases commonly encountered during 4×4 cube solving.

These unexpected observations motivated a closer investigation into the relationship between these unusual patterns and the well-known PLL parity.

This notebook records observations made during actual cube solving. The experiments are exploratory and were originally conducted to understand these puzzling patterns. The observations presented here are independent of the particular OLL parity correction algorithm used.

The three experiments presented in this notebook illustrate three representative classes of illegal top-layer four-edge-pair configurations that were later found to account for all **ten illegal states** described in **Notebook #5**.

2. Initial Observation

After correcting the OLL parity to restore the yellow cross and subsequently applying standard 3×3 algorithms (such as *Sune*, *A-Perm*, and *Y-Perm*), all corners were solved. However, the four top-layer edge pairs occasionally remained in apparently abnormal configurations.

These configurations appeared unrelated to the familiar opposite-edge PLL parity cases.

This naturally raised the following question:

Are these unusual patterns genuinely new parity cases, or are they simply different visible representations of the same underlying PLL parity?

The following three representative experiments were designed to investigate this question.

3. Experiment 1 — A Zero-Aligned Illegal State

Experiment 1 — Transforming a 0-Aligned Illegal State into the Golden Pattern

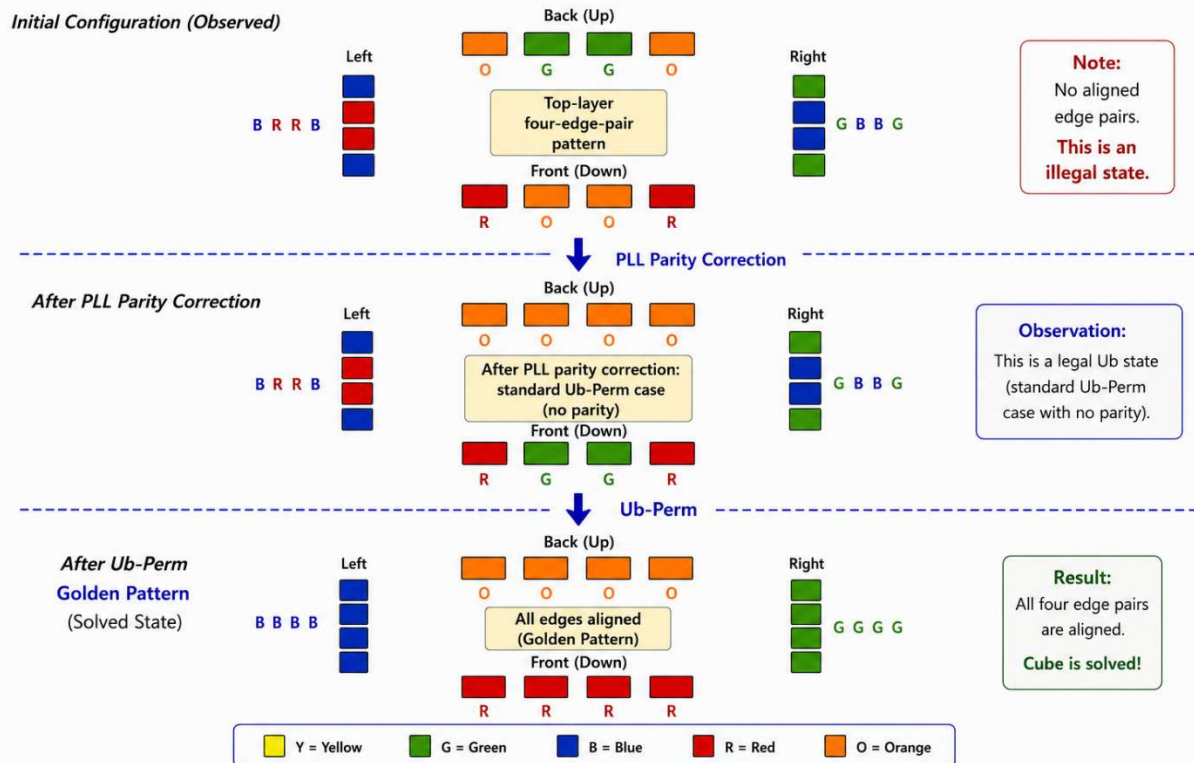


Figure 1.1. A zero-aligned illegal state.

Observed sequence is illustrated in Figure 1.1.

Observation

Applying the standard PLL parity correction transformed the illegal state into an ordinary 3×3 *Ub-Perm* case.

This experiment demonstrates that one class of illegal four-edge-pair configurations can be interpreted as another representation of the underlying PLL parity.

4. Experiment 2 — An Illegal State Transformed by *Z-Perm*

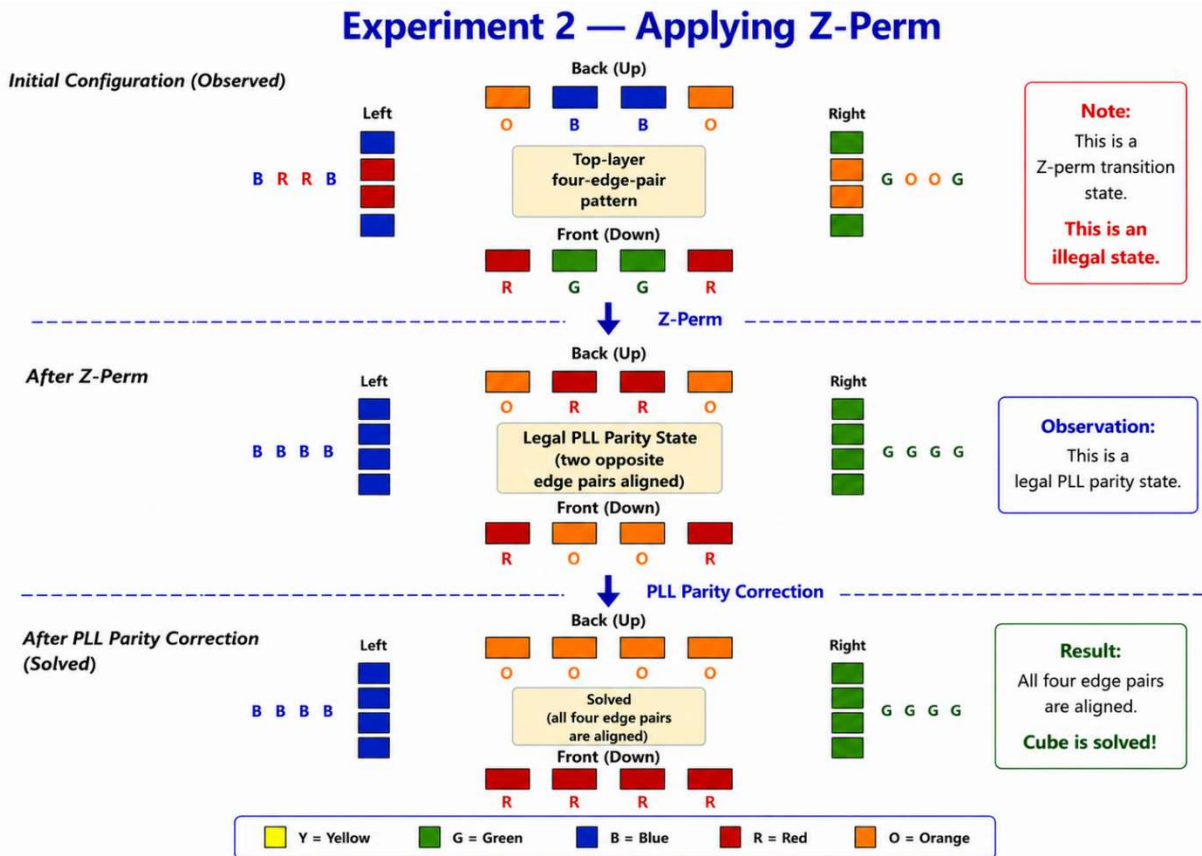


Figure 1.2. An illegal state transformed by *Z-Perm*.

Observed sequence is illustrated in Figure 1.2.

Observation

Applying *Z-Perm* transformed the illegal configuration into a familiar legal opposite-edge PLL parity state.

After the standard PLL parity correction, the cube was solved immediately without requiring any additional PLL algorithm.

5. Experiment 3 — A Two-Adjacently-Aligned Illegal State

Experiment 3 — Transforming a Two-Adjacently-Aligned Illegal State into the Golden Pattern

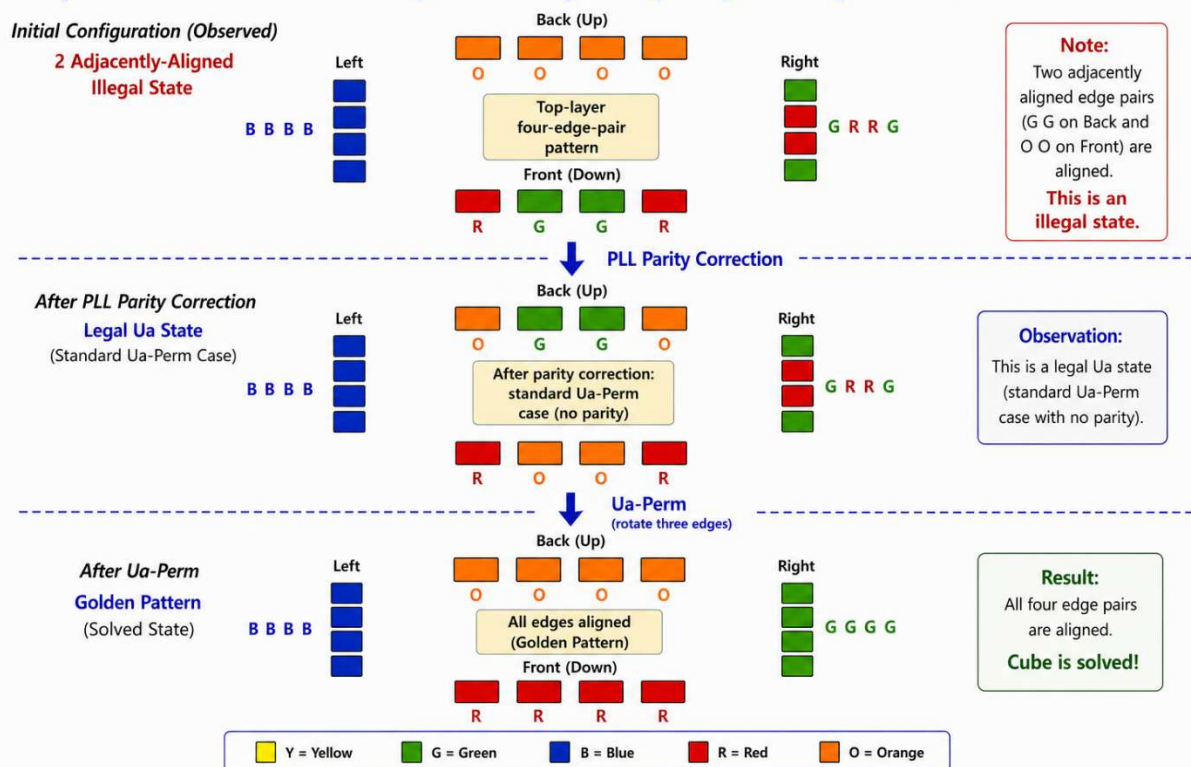


Figure 1.3. A two-adjacently-aligned illegal state.

Observed sequence is illustrated in Figure 1.3.

Observation

Applying the standard PLL parity correction transformed this illegal configuration into an ordinary 3×3 *Ua-Perm* case.

This experiment shows that another seemingly unrelated illegal configuration also corresponds to the same underlying parity phenomenon.

Together, Experiments 1–3 illustrate three representative classes of illegal four-edge-pair configurations.

6. Hypothesis

The observed illegal four-edge-pair configurations do not represent new types of parity.

Instead, they appear to be different visible representations of the same underlying PLL parity.

Standard 3×3 PLL algorithms, such as *Z-Perm*, do not eliminate the underlying parity. Instead, they may transform one visible representation of the parity into another.

A PLL parity correction removes the underlying parity and may leave either a legal *Ua-Perm* or a legal *Ub-Perm* case, depending on which opposite edge pair is selected during the correction. The two choices are symmetric.

7. Discussion

The three experiments consistently suggest that apparently different illegal four-edge-pair configurations may share the same underlying parity.

During the reduction phase of a 4×4 cube, wide-layer moves allow parity states to be created or eliminated.

Once reduction is complete, however, ordinary 3×3 algorithms appear to preserve the underlying parity while changing only its visible representation.

These observations eventually led to the broader structural classification presented in **Notebook #5**, where all twenty-four top-layer four-edge-pair patterns are classified into six structural groups, and the ten illegal states are organized into two deterministic transition paths.

Although the present notebook is based on experimental observations rather than formal proofs, the results consistently support this interpretation and motivate the structural framework presented in **Notebook #5**.

8. Open Questions

Several interesting questions remain for future investigation.

- Can these transformations be explained using permutation-group or graph-theoretic analysis?
 - Can illegal configurations be recognized before entering the 3×3 solving phase?
-

9. Lessons Learned

- Not every unusual four-edge-pair pattern represents a new parity.
 - Different illegal configurations may represent the same underlying parity.
 - PLL parity correction may lead to either *Ua-Perm* or *Ub-Perm*, depending on which opposite edge pair is selected during the correction.
 - Ordinary 3×3 algorithms appear to change only the visible representation of the parity.
 - These experimental observations motivated the structural framework developed later in **Notebook #5**.
-

Engineering Perspective

Several seemingly unrelated four-edge-pair configurations were found to share a common underlying parity behavior. These experimental observations eventually led to a structural classification of all twenty-four top-layer four-edge-pair patterns and their deterministic state transitions. Although still exploratory, this engineering perspective provides a logical interpretation of several puzzling parity phenomena encountered during 4×4 cube solving.

The Wang Method Series

Notebook #2 (Version 1.0)

Understanding the Last Three Edge Pairs in the Yau 3-2-3 Pairing Strategy

Revealing the Hidden Four-Wing Cycle

Laung-Terng (LT) Wang

August 1, 2026

1. Motivation

The Yau Method is one of the most widely adopted methods for solving the 4×4 Rubik's Cube.

A hallmark of the method is its **3-2-3 edge-pairing strategy**, in which three edge pairs are solved first, followed by two middle edge pairs, leaving only the last three edge pairs to complete.

Although numerous tutorials explain the procedures and algorithms, relatively little attention has been given to **why this strategy works so effectively**.

During a recent study of the last three edge pairs, an unexpected structural pattern emerged.

Rather than appearing random, the remaining four free wings exhibited an underlying cyclic relationship.

This notebook documents that observation.

2. Observation

After completing

- the first three edge pairs, and
- the middle two edge pairs,

one aligned edge pair remains intact.

The remaining **four free wings** determine how the last three edge pairs will be completed. Initially these four wings appear to be randomly arranged. However, after a simple slice movement, an unexpected hidden structure becomes visible. This observation motivated the following experiment.

3. Experiment 4 — Revealing the Hidden Four-Wing Cycle

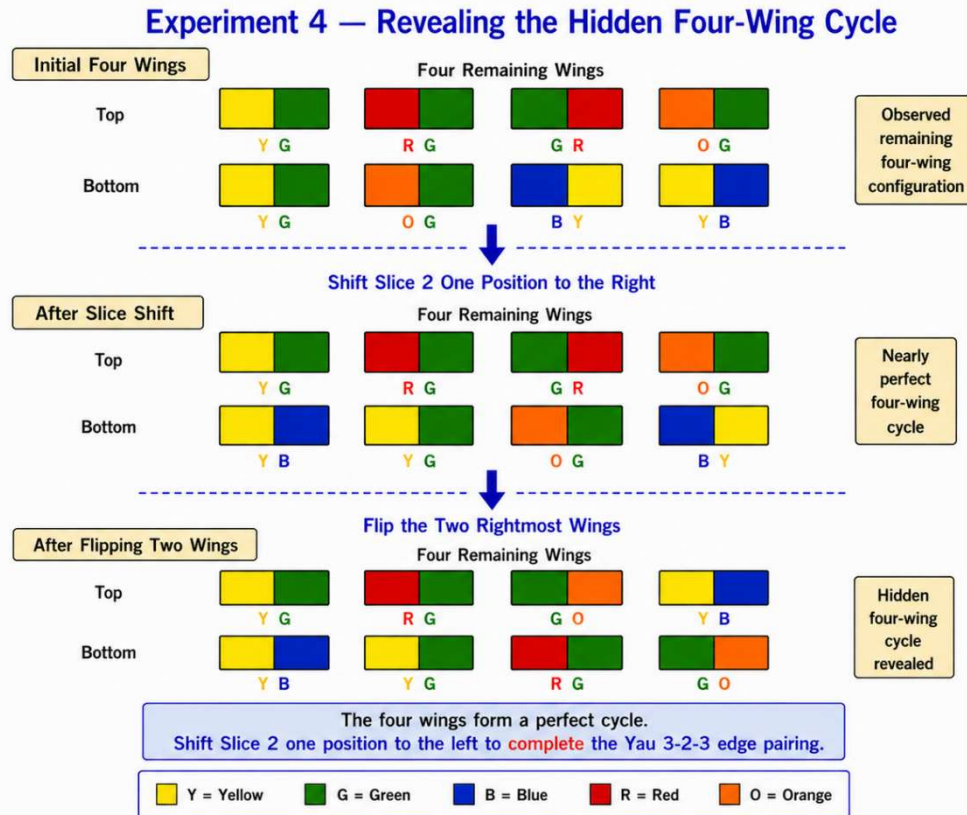


Figure 2.1. Revealing the hidden four-wing cycle.

Observed sequence is illustrated in Figure 2.1.

Initial Four Wings



Shift Slice 2 One Position to the Right



Nearly Perfect Four-Wing Cycle



Flip the Two Rightmost Wings



Perfect Four-Wing Cycle

Observation

Initially, the remaining four free wings appear to have no obvious relationship. This hidden cyclic relationship is not immediately visible in the original configuration.

After shifting Slice 2 by one position, however, they almost form a complete cyclic ordering. Only the orientations of the two rightmost wings remain incorrect.

After flipping these two wings, the four wings become a perfect cycle. The remaining edge pairs can then be completed naturally using the standard Yau procedure. No parity algorithm is involved.

4. Hypothesis

Hypothesis

The effectiveness of the Yau 3-2-3 pairing strategy may arise from the fact that, after the first nine edge pairs have been completed, the **remaining four free wings** exhibit a highly constrained cyclic structure.

A simple slice movement appears to reveal this hidden cyclic structure rather than create it.

5. Discussion

This observation provides an engineering interpretation of the Yau Method. The slice movement should not be viewed merely as another solving algorithm. Instead, it appears to expose an underlying organization of the remaining four free wings. Once this cyclic relationship becomes visible, only two wing flips are required before the final edge pairs can be completed naturally.

From this viewpoint, the remaining four free wings are not randomly arranged. Instead, they appear to follow a highly constrained cyclic relationship that becomes visible after a simple slice shift.

Although this observation is based on a limited number of experiments, it offers a possible structural explanation for why the final stage of the Yau 3-2-3 pairing strategy is considerably simpler than it first appears.

6. Open Questions

Several interesting questions remain for future investigation.

- Does every Yau 3-2-3 pairing case reveal a similar hidden four-wing cycle?
 - Can the required slice movement always be predicted?
 - How many different four-wing cycle structures exist?
 - Can these cycles be characterized mathematically using permutation-group theory?
 - Can this structural interpretation lead to simpler or more intuitive solving strategies?
-

7. Lessons Learned

- The last three edge pairs are more structured than they initially appear.
 - Slice movements appear to reveal an existing structure rather than create one.
 - A hidden four-wing cycle may underlie the final stage of the Yau 3-2-3 pairing strategy.
 - Understanding the cube's structure may be more valuable than memorizing additional algorithms.
-

Engineering Perspective

The effectiveness of the Yau 3-2-3 pairing strategy may arise not only from its algorithms, but also from the hidden structure of the remaining four free wings. Once this cyclic relationship is recognized, the final stage of edge pairing becomes considerably easier to understand.

The Wang Method Series

Notebook #3 (Version 1.0)

Structural Classification of the Last Three Edge Pairs

A Study of Edge Isomorphism and Hierarchical Pattern Recognition

Laung-Terng (LT) Wang

August 1, 2026

1. Motivation

When solving the 4×4 Rubik's Cube using the Yau method, the last three edge pairs (**L3E**) often appear in a variety of seemingly unrelated arrangements. At first glance, these configurations appear to require numerous special cases to be memorized.

This notebook shows that the apparent complexity is largely an illusion.

By treating the two edges as **isomorphic**, the remaining three edge pairs exhibit a surprisingly simple structural organization. Rather than viewing each configuration as an independent case, the solver only needs to recognize a small number of structural patterns.

This viewpoint leads naturally to a hierarchical classification of the remaining three edge pairs. More importantly, it provides a unified framework for understanding why different edge-pairing situations often require similar solving strategies, and why certain parity signatures may already become visible during edge pairing.

The purpose of this notebook is not merely to classify the remaining edge-pairing configurations, but to reveal the underlying structural relationships that connect them.

2. A Simpler Way to Understand the Last Three Edge Pairs

The last three edge pairs may look confusing at first.

Many cases seem different.

Do we really need to remember every one of them?

No.

Many of these cases differ only in the order of their colors. Once we ignore the color order, many "different" cases become the same.

This notebook will show you how to recognize these hidden patterns.

Let's look at a simple example Figure 3.1 together — Same Edge, Different Looks.

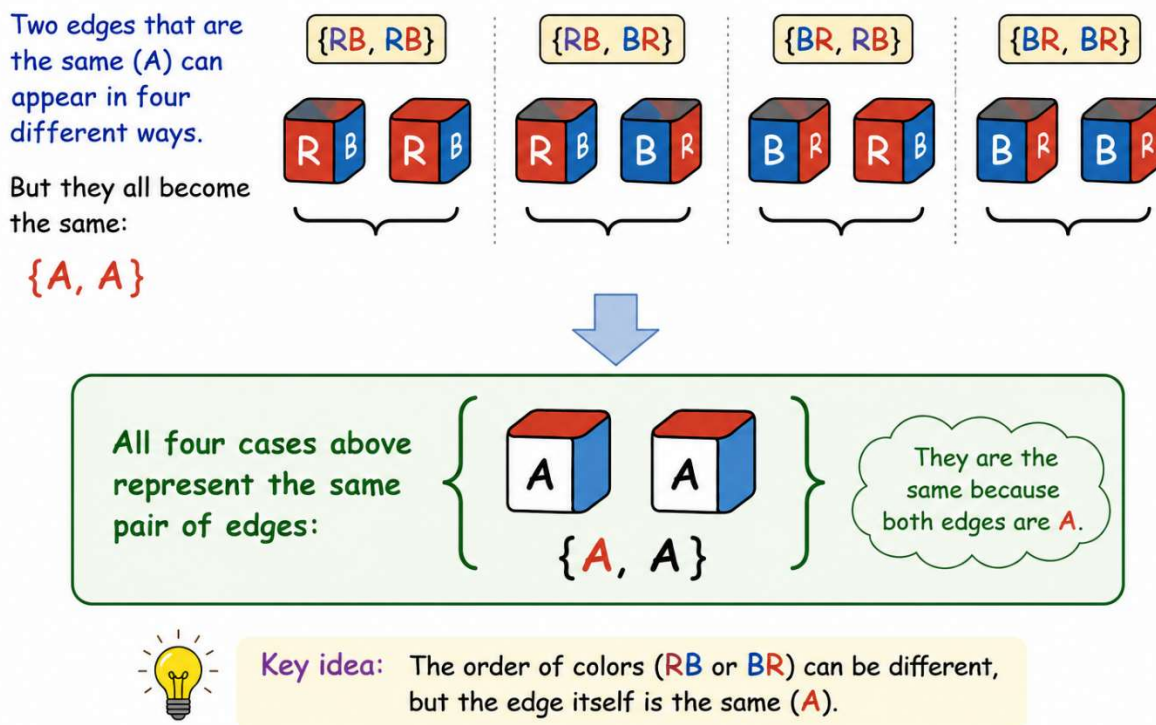


Figure 3.1. Same edges, different looks.

A Red–Blue edge may appear as **RB** or **BR**.

Although the color order is different, it is still the same edge.

So we simply call both appearances **A**.

Now look at two **A** edges.

As shown in Figure 3.1, they can appear in four different ways.

Although these four appearances look different, they all represent the same edge pair:

$\{A, A\}$

Instead of remembering every color order, we simply use the letters **A**, **B**, **C**, and **D** to represent the four edge types.

The letter (**A**, **B**, **C**, or **D**) is called the **Edge Identity**.

The color order (such as **RB** or **BR**) is called the **Edge Orientation**.

Although **RB** and **BR** look different, they represent the same edge.

Treating them as the same edge is called **Edge Isomorphism**.

From now on, we will use **Edge Identity** to classify L3E patterns. **Edge Orientation** will be used later when we study parity.

This structural viewpoint also provides the foundation for the parity analysis presented later in this notebook.

3. Hierarchical Structural Recognition

The recognition process begins by inspecting the first two columns. If exactly one pair of identical symbols is present, the configuration immediately belongs to the first cross structural family (Cases 1–4).

If this condition is not satisfied, attention is shifted to the remaining two columns. When an aligned edge pair is present, the orientation of the alignment determines the corresponding structural family. A vertically aligned edge pair belongs to Cases 6–9, whereas a horizontally aligned edge pair belongs to Cases 11–14.

If neither condition is satisfied, the remaining configuration belongs to the diagonal family (Cases 16–19).

Consequently, the sixteen structural cases are recognized through a sequence of simple visual observations rather than by memorizing individual configurations. This hierarchical recognition process greatly reduces the cognitive effort required during edge pairing and makes the structural relationships among different cases much more apparent.

From an educational perspective, this recognition strategy is particularly attractive because beginners can quickly identify the appropriate structural family after only a few seconds of inspection. The emphasis shifts naturally from memorizing numerous cases to recognizing a small number of recurring structural patterns.

4. Structural Classification

Based on the concept of edge isomorphism and the hierarchical recognition strategy introduced above, the sixteen structural cases can be organized into four structural families, each representing a distinct structural signature, as illustrated in Figures 3.2 through 3.5.

The following four figures summarize the four structural families together with their corresponding Ideal Cases (Cases 5, 10, 15, and 20). Each figure illustrates the minimum structural and orientation corrections required to transform every structural configuration into its associated Ideal Case

Family I – Cross Structure (Cases 1–4)

The first family is recognized by a unique cross structure. After the aligned edge pair A is shifted to its target position, the other diagonal edge pair B within the first two columns forms a distinctive cross with edge pair A. This cross pattern is unique to Family I.

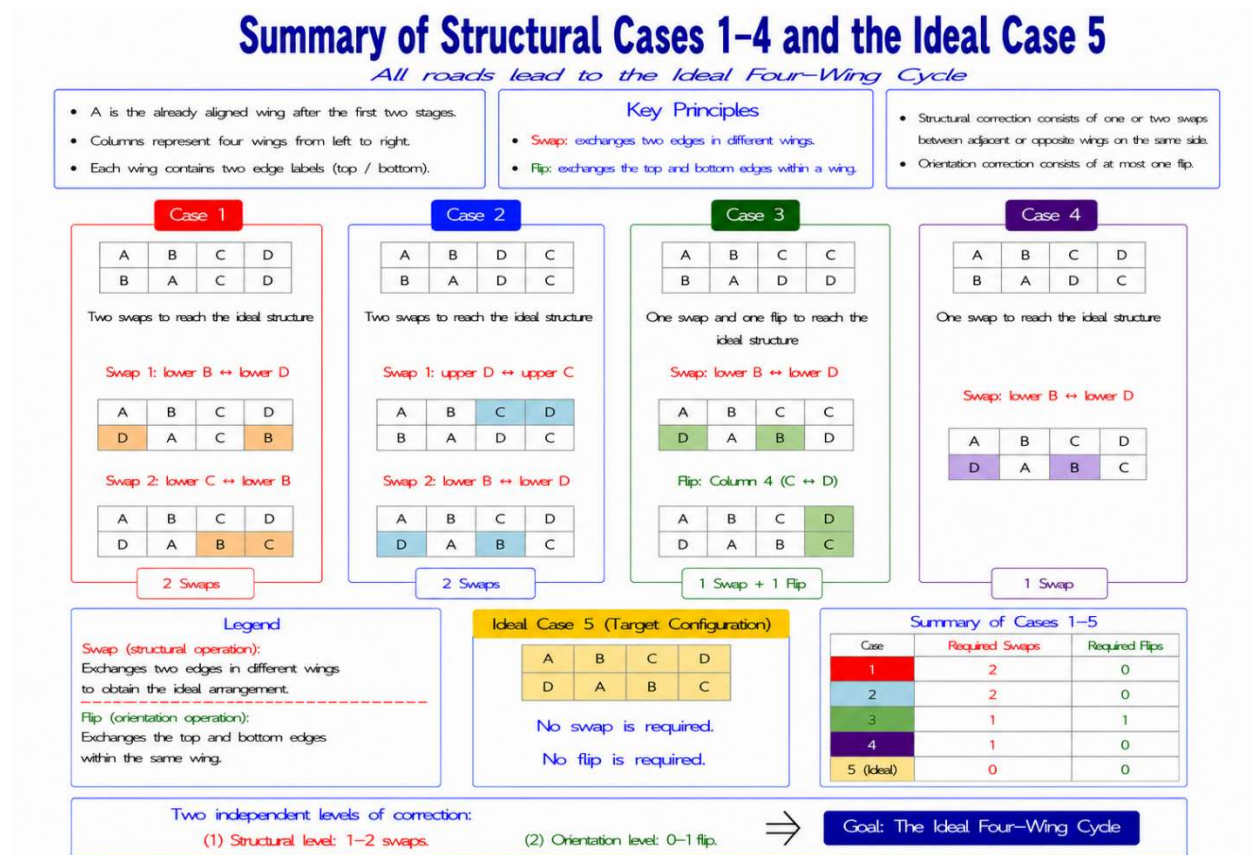


Figure 3.2. Cases 1–4 and the Ideal Case 5.

Family II – Vertical Alignment (Cases 6–9)

The second family is recognized by a vertically aligned identical edge pair within the rightmost two columns. Structural correction requires only one swap, while orientation correction requires only one flip.

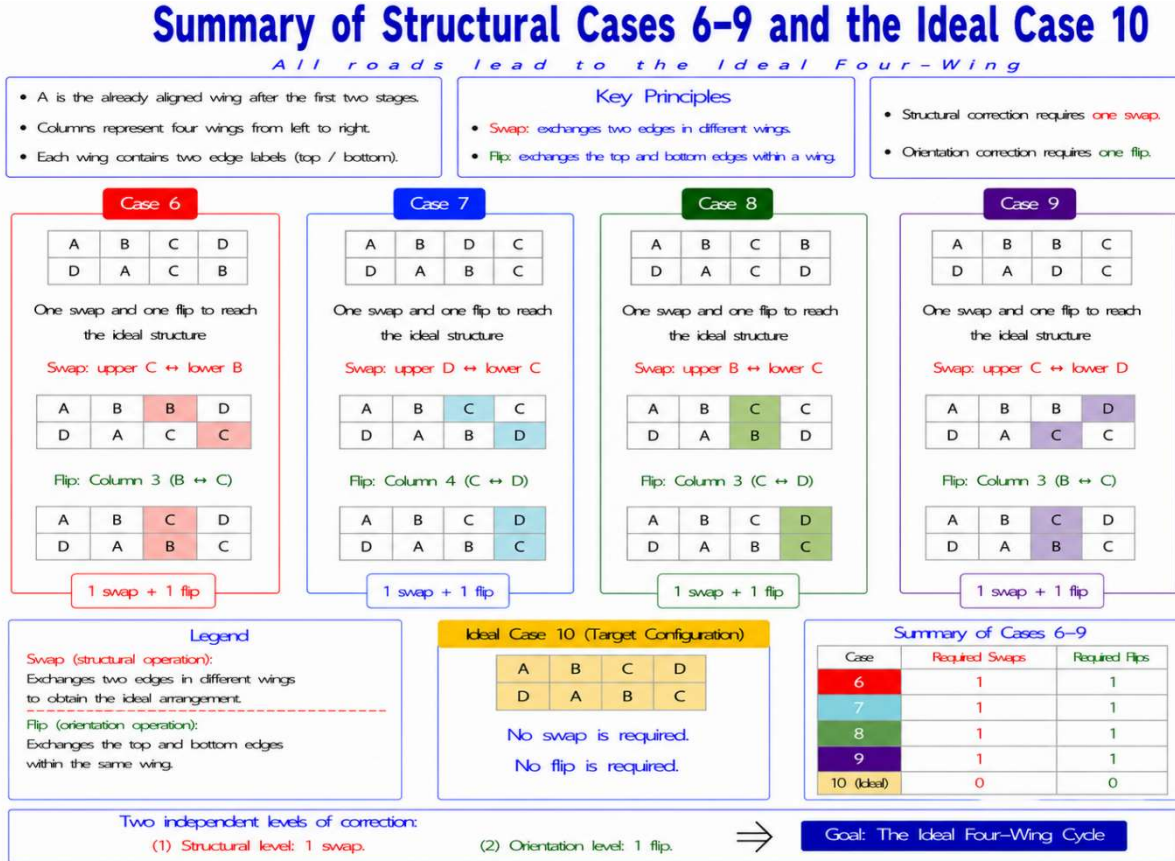


Figure 3.3. Cases 6–9 and the Ideal Case 10.

Family III – Horizontal Alignment (Cases 11–14)

The third family is recognized by a horizontally aligned identical pair within the remaining two rows. Similar to the previous family, structural and orientation corrections remain independent, allowing the required operations to be analyzed separately.

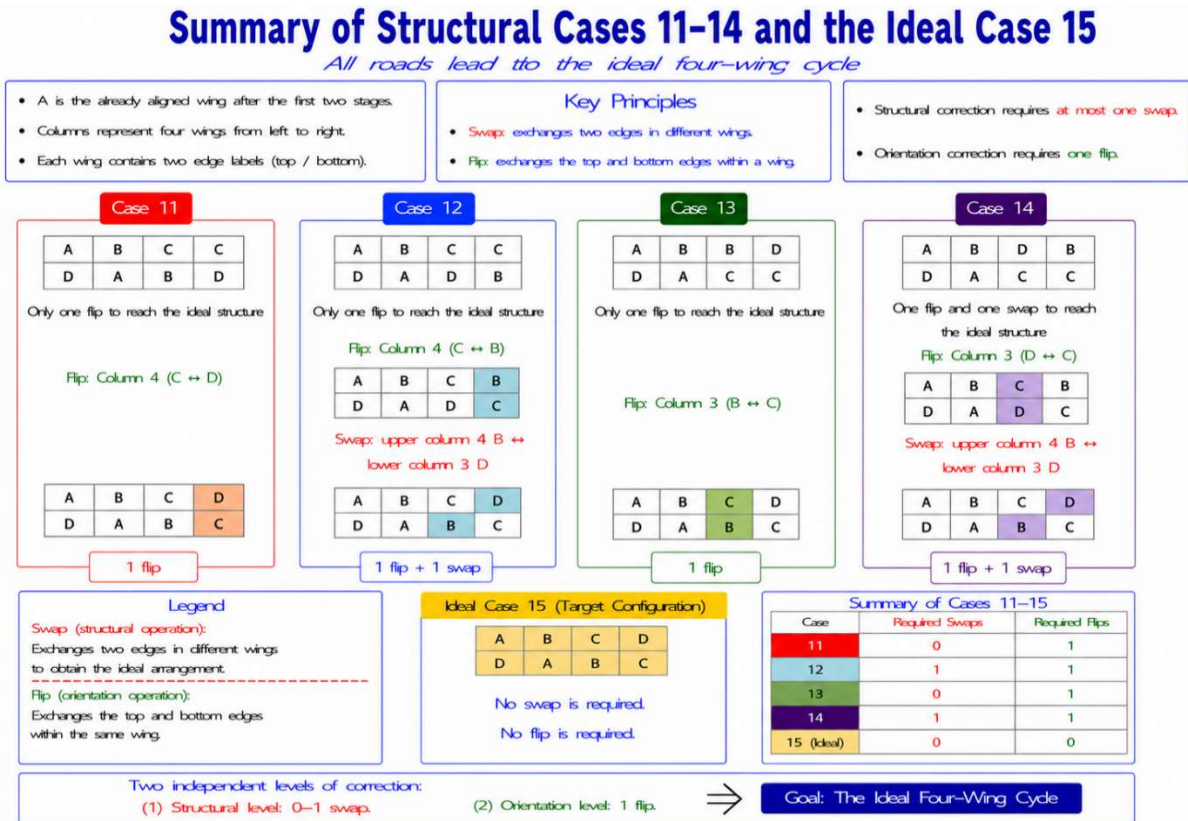


Figure 3.4. Cases 11–14 and the Ideal Case 15.

Family IV – Diagonal Alignment (Cases 16–19)

The fourth family is recognized by diagonally aligned identical symbols. Although this family looks different from the previous ones, the same structural principles continue to apply. Structural correction requires at most one swap, whereas orientation correction requires at most two flips.

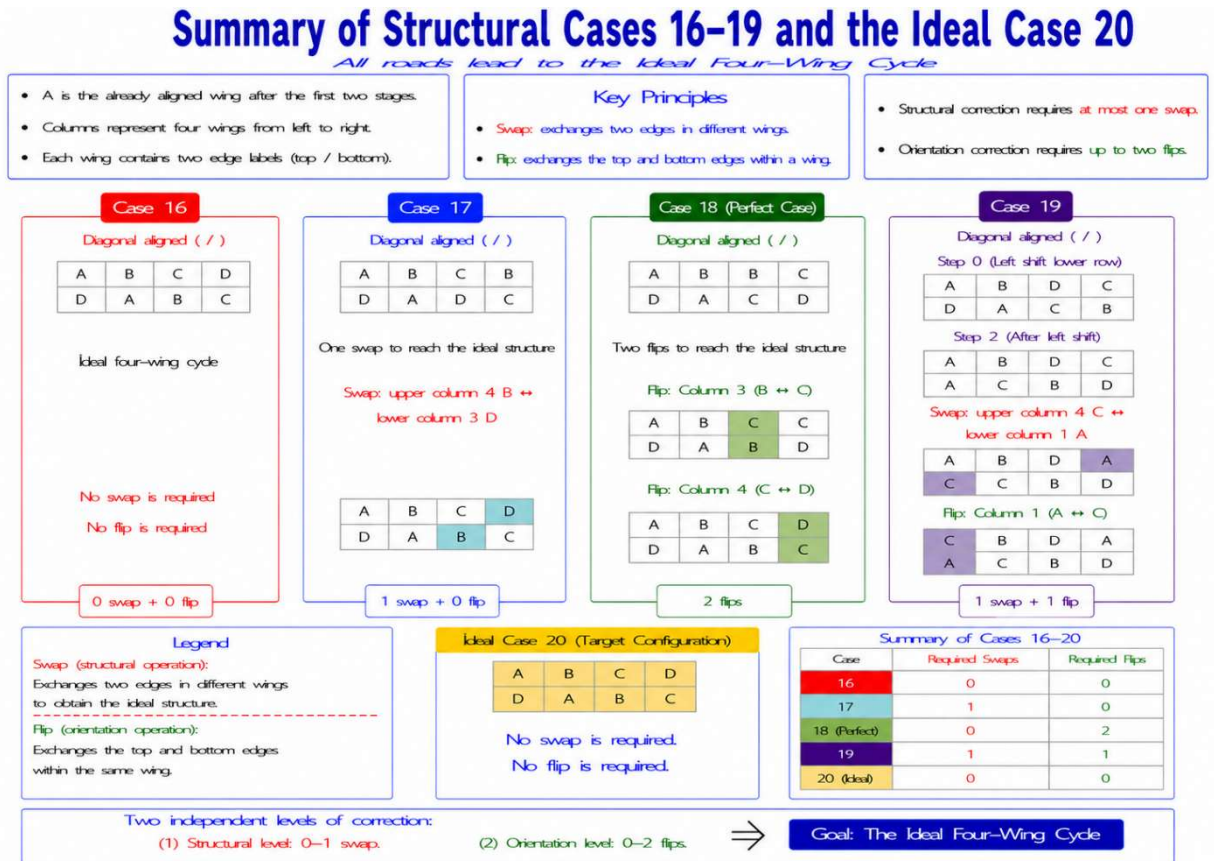


Figure 3.5. Cases 16–19 and the Ideal Case 20.

Although several of these cases in Cases 1–4 initially appear to require one or two structural horizontal swaps, equivalent transformations can often be achieved more efficiently by exploiting diagonal swaps among the four wings. This implementation perspective demonstrates that different structural configurations may share common solving strategies despite their different visual appearances.

Rather than treating the last three edge-pair configurations as unrelated cases, they can be grouped into a small number of structural families based on common structural principles.

Figure 3.6 illustrates that this structural viewpoint not only simplifies classification but also leads naturally to more efficient solution strategies.

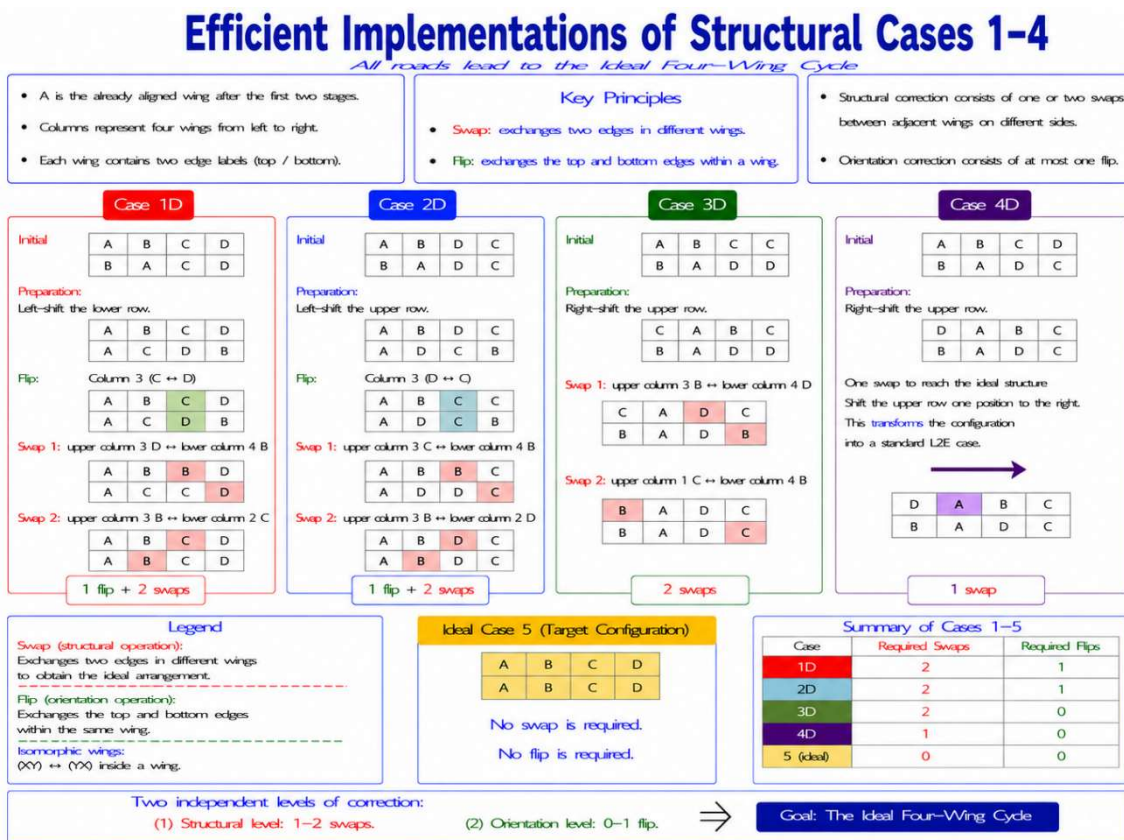


Figure 3.6. Efficient implementations of Cases 1–4.

5. Discussion

The hierarchical structural classification presented in this notebook is a direct consequence of the concept of edge isomorphism. Once the two wings belonging to the same edge are regarded as structurally equivalent, many apparently different edge-pairing configurations become members of the same structural family.

More importantly, the classification emphasizes **recognition rather than memorization**. This significantly reduces the cognitive load on beginners. Instead of remembering numerous independent cases, the solver identifies a small number of structural signatures and then applies the corresponding correction strategy.

The resulting framework also separates structural correction from orientation correction. Structural errors are corrected by swaps, whereas orientation errors are corrected by flips. Because these two types of corrections are largely independent, both the analysis and the implementation become considerably simpler.

From an engineering viewpoint, the classification transforms edge pairing from a collection of isolated cases into a unified structural problem. The emphasis is placed on understanding the underlying relationships among the remaining wings rather than memorizing individual configurations.

6. Parity Signatures during Edge Pairing

The concept of edge isomorphism also leads to an interesting observation regarding parity during edge pairing.

During edge reduction, the remaining wing configuration may already exhibit an OLL-parity signature. Because paired wings behave as equivalent representations of reduced edges, one apparent edge pair may remain reversed with respect to the others. This orientation inconsistency corresponds naturally to the well-known OLL parity encountered after edge reduction.

In contrast, PLL parity cannot yet be revealed during edge pairing.

PLL parity concerns the permutation of the completed reduced edges. Before all edge pairs have been completed, the remaining free wings can still be rearranged, allowing incomplete permutation cycles to be completed naturally. Consequently, an apparent exchange between two edge pairs does not yet constitute a genuine PLL-parity state.

This structural interpretation provides a simple explanation for an observation familiar to many experienced solvers: OLL parity may already become visible during edge pairing, whereas PLL parity becomes meaningful only after the reduced-edge permutation has been completely established.

7. Preliminary Observations and Open Questions

Based on the experiments completed so far, only one OLL-parity signature has been observed during edge pairing.

No example containing two simultaneous OLL-parity signatures has yet been found. Although this observation suggests that the underlying wing-permutation structure may restrict the cube to at most one OLL-parity signature during edge pairing, no mathematical proof is currently available.

Likewise, no PLL-parity signature has ever been observed before edge reduction has been completed. This observation is consistent with the structural interpretation presented in this notebook, although a rigorous theoretical explanation remains to be established.

Several interesting questions therefore remain open.

- Can the restriction of at most one OLL-parity signature be proved mathematically?
- Can a counterexample containing two simultaneous OLL-parity signatures exist?
- Can the structural classification be derived directly from permutation-group theory?
- Can the concept of edge isomorphism be generalized to larger even-order cubes, such as the 6×6 and 8×8 ?

These questions suggest several promising directions for future investigation.

8. Lessons Learned

Treating two edges as isomorphic provides a remarkably simple framework for understanding the last three edge pairs.

Rather than viewing edge pairing as a large collection of unrelated special cases, the remaining configurations can be recognized hierarchically through a small number of structural signatures. This viewpoint shifts the emphasis naturally from memorization to structural recognition and provides a much simpler way to understand the underlying structural relationships among the remaining edge pairs.

The resulting classification not only organizes the remaining edge-pairing configurations into four structural families but also explains why structurally different cases frequently require similar solving strategies.

The same structural viewpoint also offers a natural interpretation of parity during edge pairing. OLL-parity signatures may already become visible because orientation inconsistencies can already be observed among the partially reduced edges, whereas PLL parity cannot yet emerge because the reduced-edge permutation has not been completely established.

Although many theoretical questions remain open, edge isomorphism provides a simple engineering abstraction that unifies and greatly clarifies the structural behavior of the remaining edge pairs.

Engineering Perspective

Engineering often begins by replacing apparent complexity with an appropriate abstraction.

In this notebook, edge isomorphism serves as that abstraction.

Once the two wings belonging to the same edge are regarded as structurally equivalent, the remaining edge pairs no longer appear as numerous unrelated cases. Instead, they become members of a small number of recognizable structural families governed by common structural principles.

From this viewpoint, edge pairing becomes an exercise in structural recognition rather than case memorization. The same abstraction also explains naturally why OLL-parity signatures may already become visible during edge pairing, whereas PLL parity cannot emerge until the reduced-edge permutation has been completely established.

This notebook therefore represents not only a classification of edge-pairing configurations but also an engineering perspective for understanding their underlying structure.

The Wang Method Series

Notebook #4 (Version 1.0)

Wang's Simple L3E Pairing Method

A Look Ahead Method for Solving the Last Three Edge Pairs of the 4×4 Rubik's Cube

Laung-Terng (LT) Wang

August 1, 2026

1. Introduction

After completing the first five edge pairs using the **3-2-3 edge pairing strategy**, the last three edge pairs (**L3E**) remain unsolved. Efficiently solving these last three edge pairs is one of the key ideas of the **Yau Method**.

This notebook presents **Wang's Simple L3E Pairing Method**, a beginner-friendly approach based on **Look Ahead**. Instead of immediately pairing the most obvious edge, which may unintentionally reduce the problem to a last two edge pairs (**L2E**) configuration, the solver first examines the complete L3E configuration and predicts the result of each possible pairing.

Two complementary search paths are used to determine whether an edge can be paired directly or should first be transformed by a **Flip** or **Swap** operation. The selected operation preserves the **L3E** structure so that the solver can continue solving all three remaining edge pairs systematically.

Figures 4.1 and 4.2 summarize the complete Look Ahead decision process.

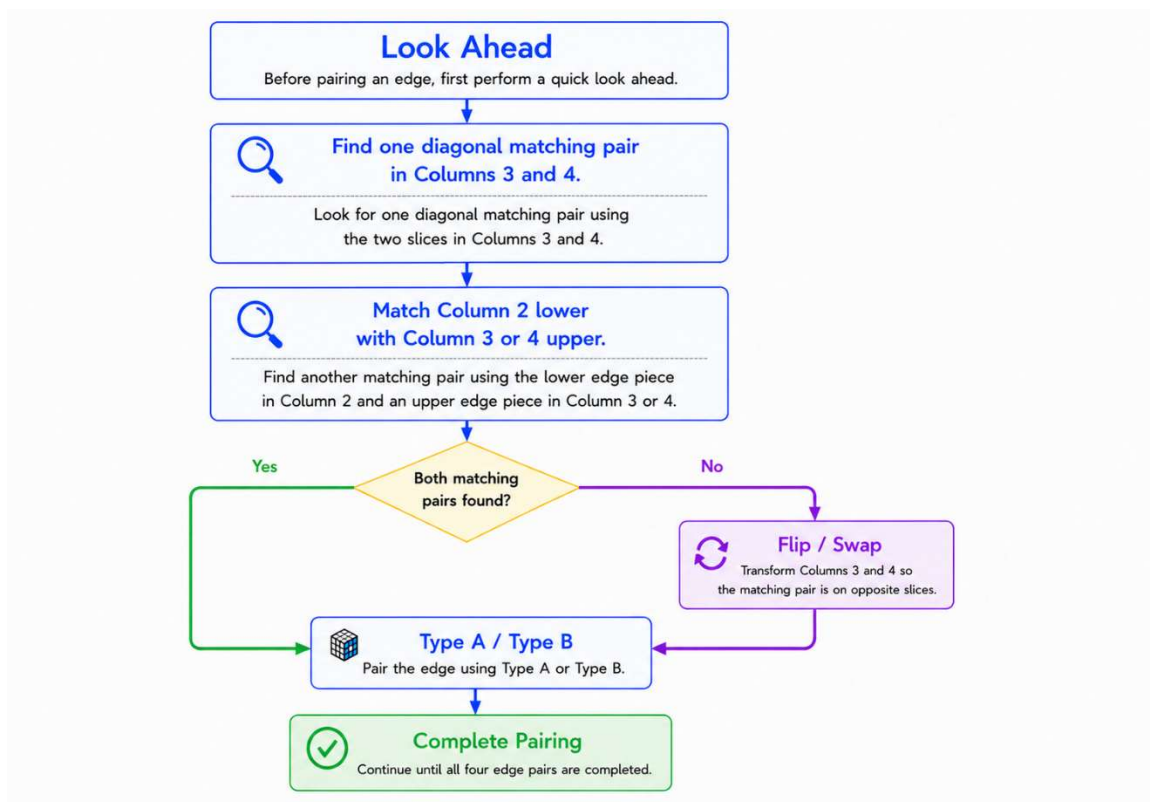


Figure 4.1. Look Ahead flowchart: Search Path 1.

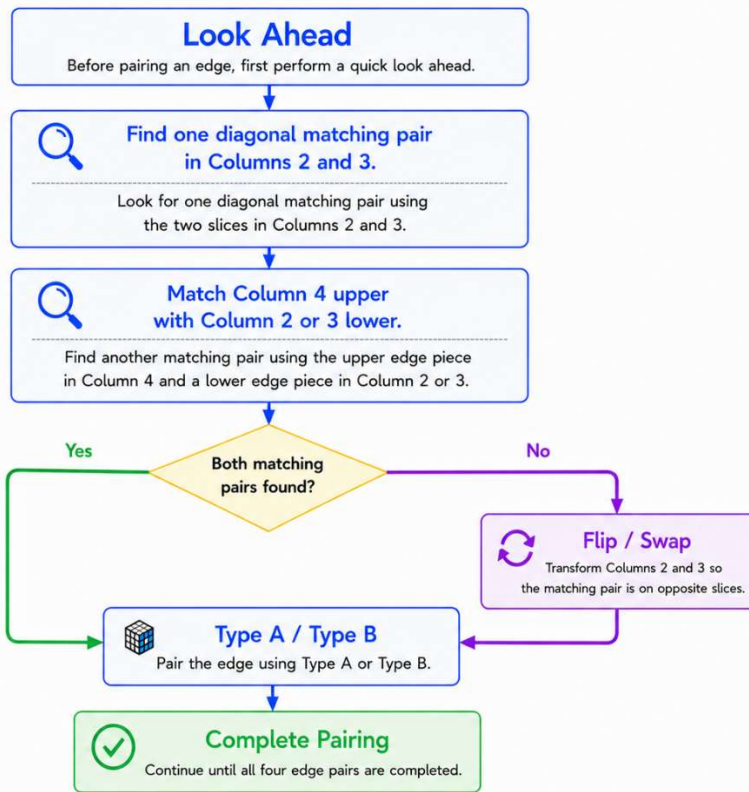


Figure 4.2. Look Ahead flowchart: Search Path 2.

2. Look Ahead

Before pairing an edge, first perform a quick **Look Ahead**.

Begin with **Search Path 1** (Figure 4.1).

- Find one diagonal matching pair in **Columns 3 and 4**.
- Then match the lower edge piece in **Column 2** with an upper edge piece in **Column 3 or 4**.

If both matching pairs are found, pair the edge directly using **Type A** or **Type B**.

If Search Path 1 is unsuccessful, try the complementary **Search Path 2** (Figure 4.2).

- Find one diagonal matching pair in **Columns 2 and 3**.
- Then match the upper edge piece in **Column 4** with a lower edge piece in **Column 2 or 3**.

Again, if both matching pairs are found, pair the edge directly using **Type A** or **Type B**.

Otherwise, perform a **Flip** or **Swap**, then complete the pairing using **Type A** or **Type B**.

This Look Ahead strategy considers the complete **L3E** configuration before any pairing operation is performed. Rather than simply solving the immediately visible edge pair, it selects a procedure that preserves the **L3E** structure and avoids unnecessarily inducing an **L2E** configuration.

3. Type A Pairing

Type A uses an upper-row shift.

Type A is one of the two fundamental pairing procedures of Wang's Simple L3E Pairing Method. The detailed five-step procedure is illustrated in Figure 4.3. Once Type A is mastered, the remaining configurations can be solved by first transforming them into Type A and then applying this procedure.

The complete Type A pairing procedure requires **five moves**.

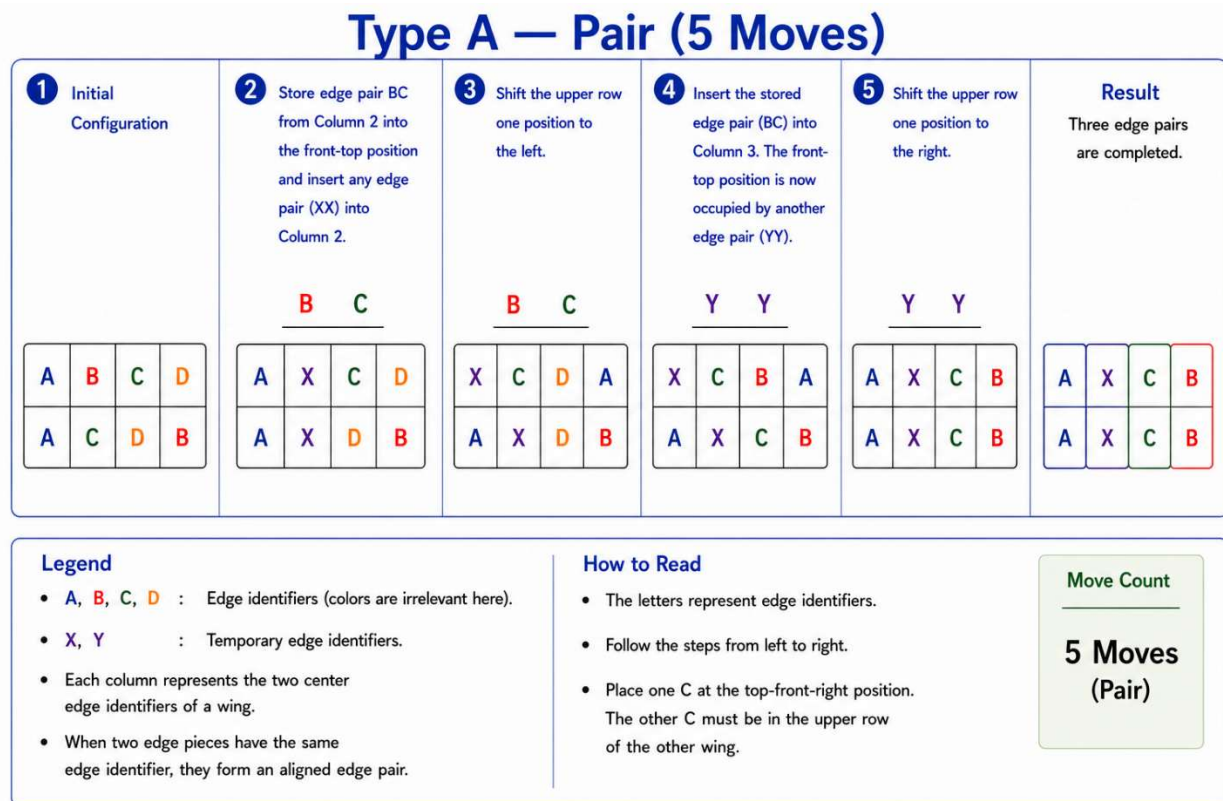


Figure 4.3: Type A — Pair (5 Moves).

4. Type B Pairing

Type B uses the same store-and-insert concept as Type A, but the lower row is shifted instead of the upper row.

Type B is the second fundamental pairing procedure of Wang's Simple L3E Pairing Method. Like Type A, it assumes that the two matching **C edge pieces** have already been positioned on opposite slices.

The complete Type B pairing procedure also requires **five moves**.

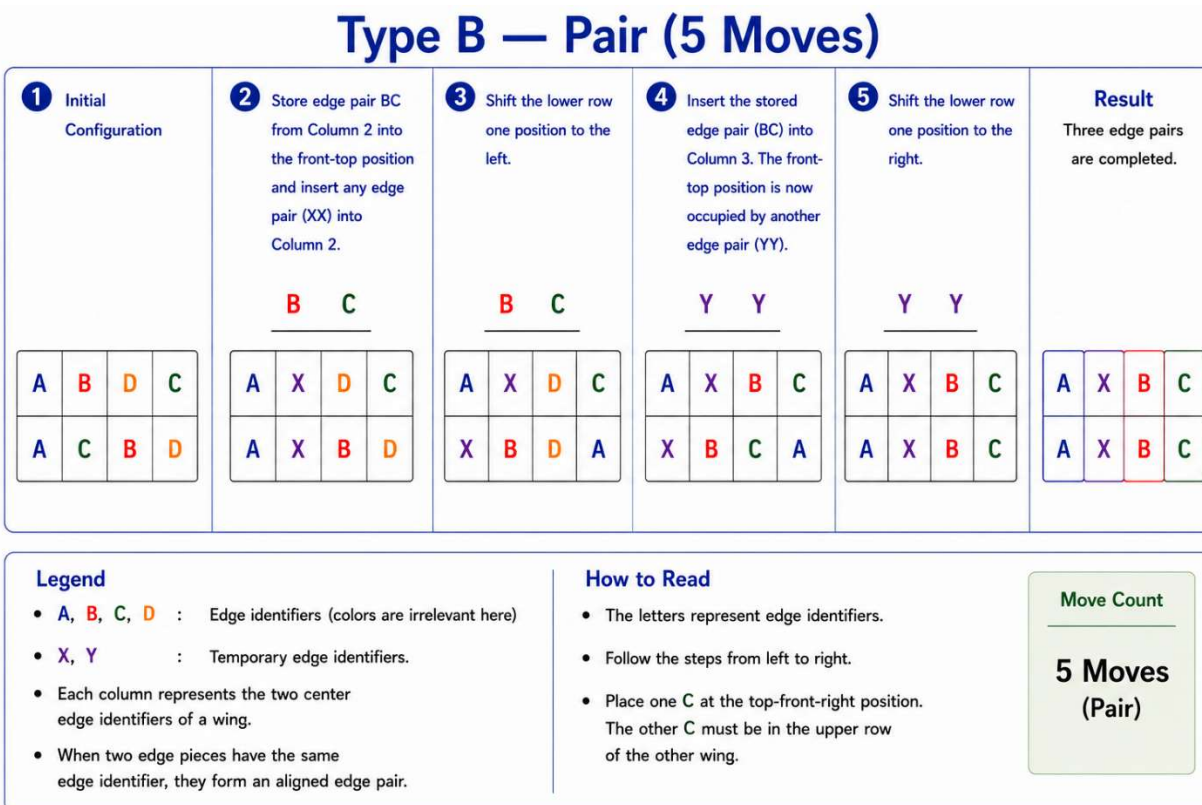


Figure 4.4: Type B — Pair (5 Moves).

5. Flip + Pair Transformations

Flip + Pair is the primary transformation used in Wang's Simple L3E Pairing Method.

A flip separates the two matching **C edge pieces** that initially occupy the same slice. The resulting configuration becomes either **Type A** or **Type B**, and can then be solved using the corresponding five-move pairing procedure.

Although a wing can be flipped in fewer moves, Wang's Simple L3E Pairing Method intentionally uses the **seven-move Flip** because it preserves the remaining prepared edge positions. After the last three edge pairs are completed, the solver can proceed directly to the 3×3 stage and begin **CFOP** with the Cross. The shortest move sequence is not always the best solution.

The complete **Flip + Pair** sequence therefore requires **twelve moves**.

5.1 Type A — Flip + Pair

For the Type A transformation, one wing is flipped so that the two matching **C** edge pieces move onto opposite slices. The resulting configuration is identical to the initial configuration of **Type A — Pair**.

The Type A pairing procedure is then applied.

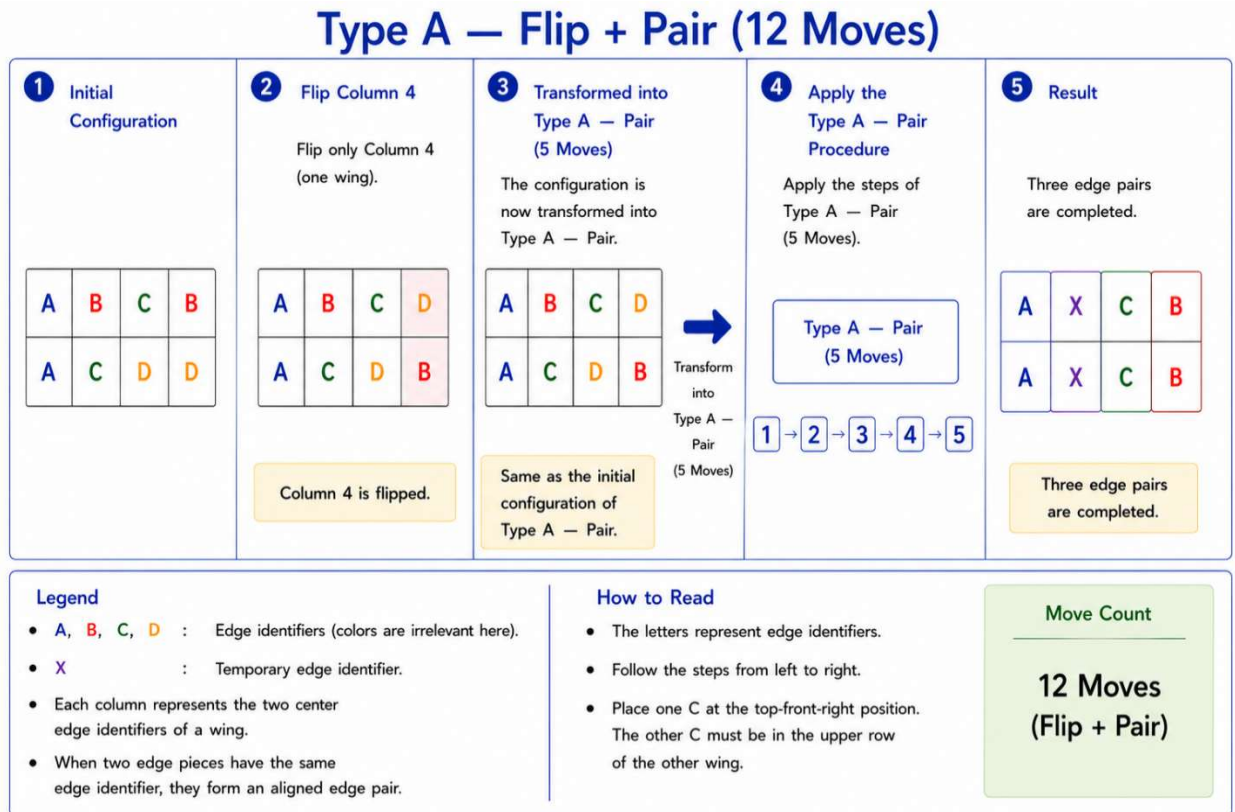


Figure 4.5: Type A — Flip + Pair (12 Moves).

5.2 Type B — Flip + Pair

For the Type B transformation, one wing is flipped to separate the two matching **C** edge pieces. The resulting configuration becomes **Type B — Pair**.

The Type B pairing procedure is then applied.

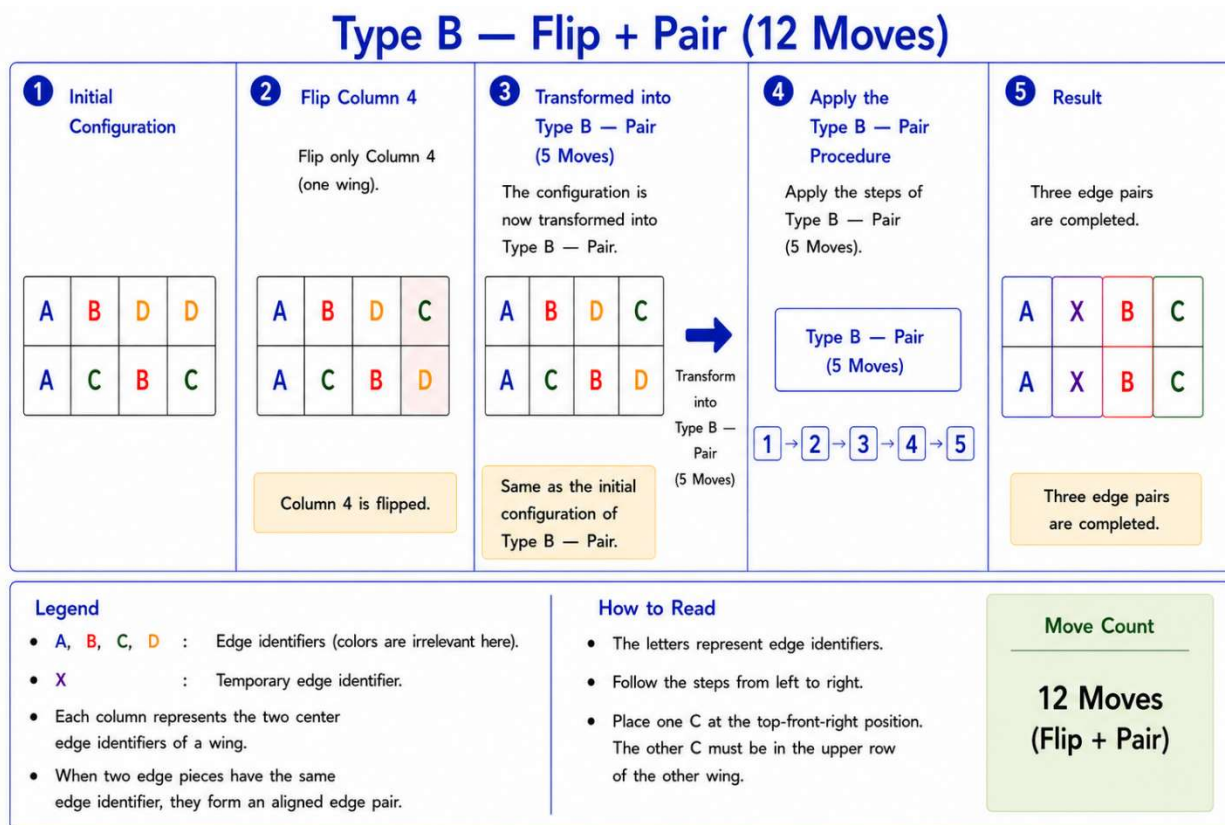


Figure 4.6: Type B — Flip + Pair (12 Moves).

6. Type C and Type D Recognition

Type C and Type D represent the two principal same-slice arrangements.

In both types, the two matching **C edge pieces** initially occupy the same slice. Therefore, the solver should not directly complete another edge pair without first transforming the configuration.

6.1 Type C — Flip + Pair

In Type C, flipping the indicated wing transforms the configuration into **Type A — Pair**. The Type A pairing procedure then completes all three edge pairs.

Thus, Type C is solved by:

Flip → Type A — Pair

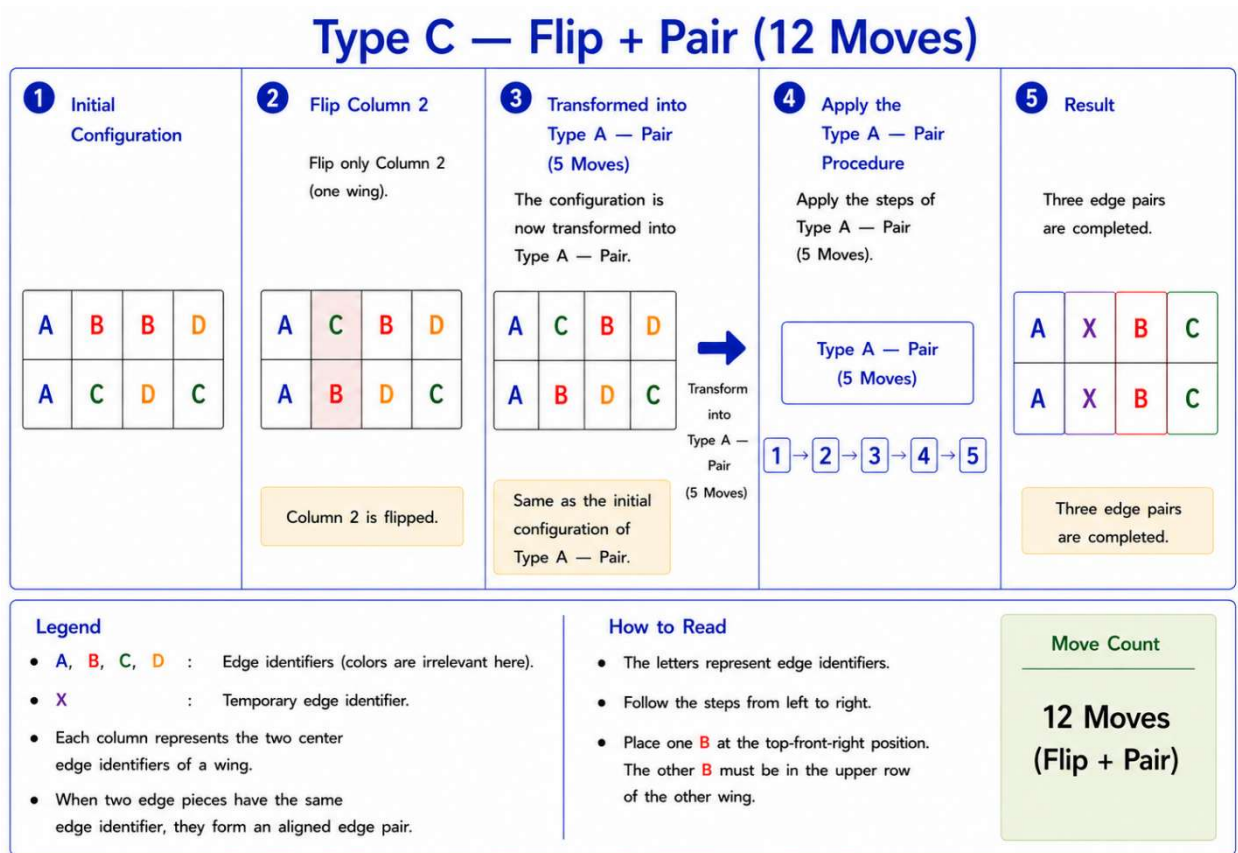


Figure 4.7: Type C — Flip + Pair (12 Moves).

6.2 Type D — Flip + Pair

In Type D, flipping the indicated wing transforms the configuration into **Type B — Pair**. The Type B pairing procedure then completes all three edge pairs.

Thus, Type D is solved by:

Flip → Type B — Pair

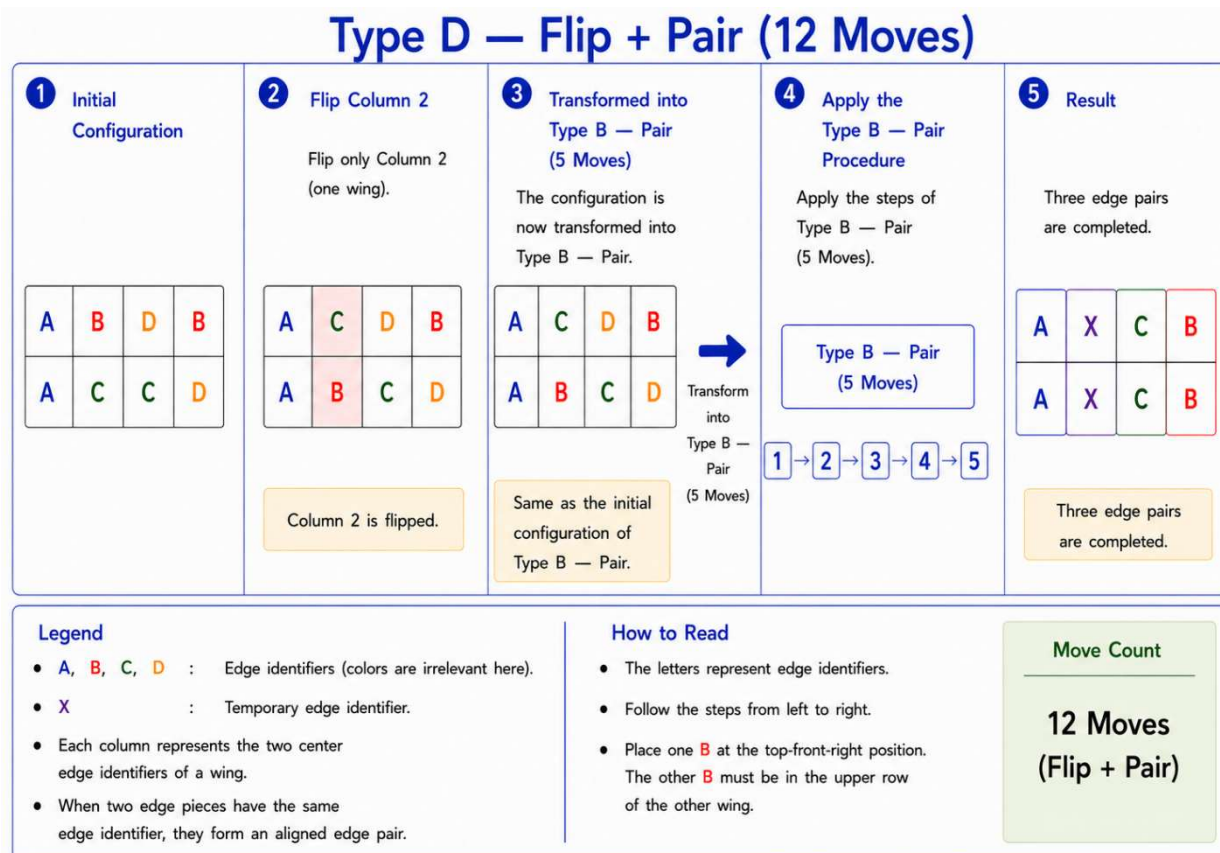


Figure 4.8: Type D — Flip + Pair (12 Moves).

No additional pairing algorithm is required for Type C or Type D.

7. Optional Swap + Pair Procedures

Flip + Pair requires only twelve moves and is therefore the preferred procedure for beginners.

However, the shortest move count does not always produce the shortest execution time for an experienced speed cuber.

When the edge pair to be preserved is already conveniently positioned at the front-top, a speed cuber may prefer to perform a Swap instead of a Flip. Although Swap + Pair requires fourteen moves, it may provide a smoother execution with fewer pauses for recognition or regripping.

Like Flip, a Swap can also be performed in fewer moves. However, Wang's Simple L3E Pairing Method intentionally uses the **nine-move Swap** because it preserves the remaining prepared edge positions. After the last three edge pairs are completed, the solver can proceed directly to the 3×3 stage and begin the standard 3×3 **CFOP** method with the Cross.

For this reason, Wang's Simple L3E Pairing Method includes **Swap + Pair** as an optional speed-cubing alternative.

The two approaches serve different purposes:

- **Flip + Pair:** fewer moves and easier for beginners to learn.
- **Swap + Pair:** an optional alternative that may be faster in actual execution when the desired edge pair is already positioned at the front-top.

The choice should therefore be based not only on move count, but also on the current cube position and the solver's execution preference.

7.1 Type A — Swap + Pair

In Type A, swapping the indicated edge pieces transforms the configuration into **Type A — Pair**. The corresponding five-move pairing procedure then completes all three edge pairs.

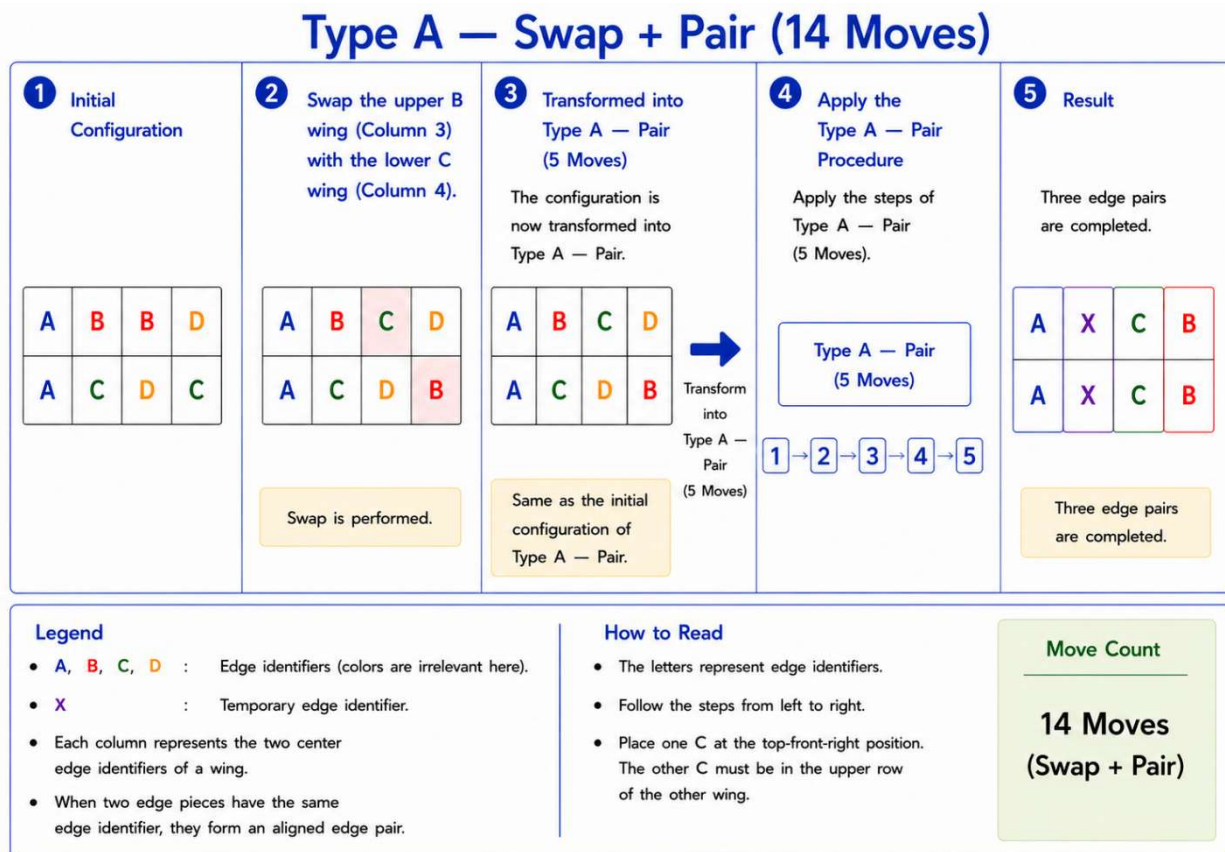


Figure 4.9: Type A — Swap + Pair (14 Moves).

7.2 Type B — Swap + Pair

In Type B, swapping the indicated edge pieces transforms the configuration into **Type B — Pair**. The corresponding five-move pairing procedure then completes all three edge pairs.

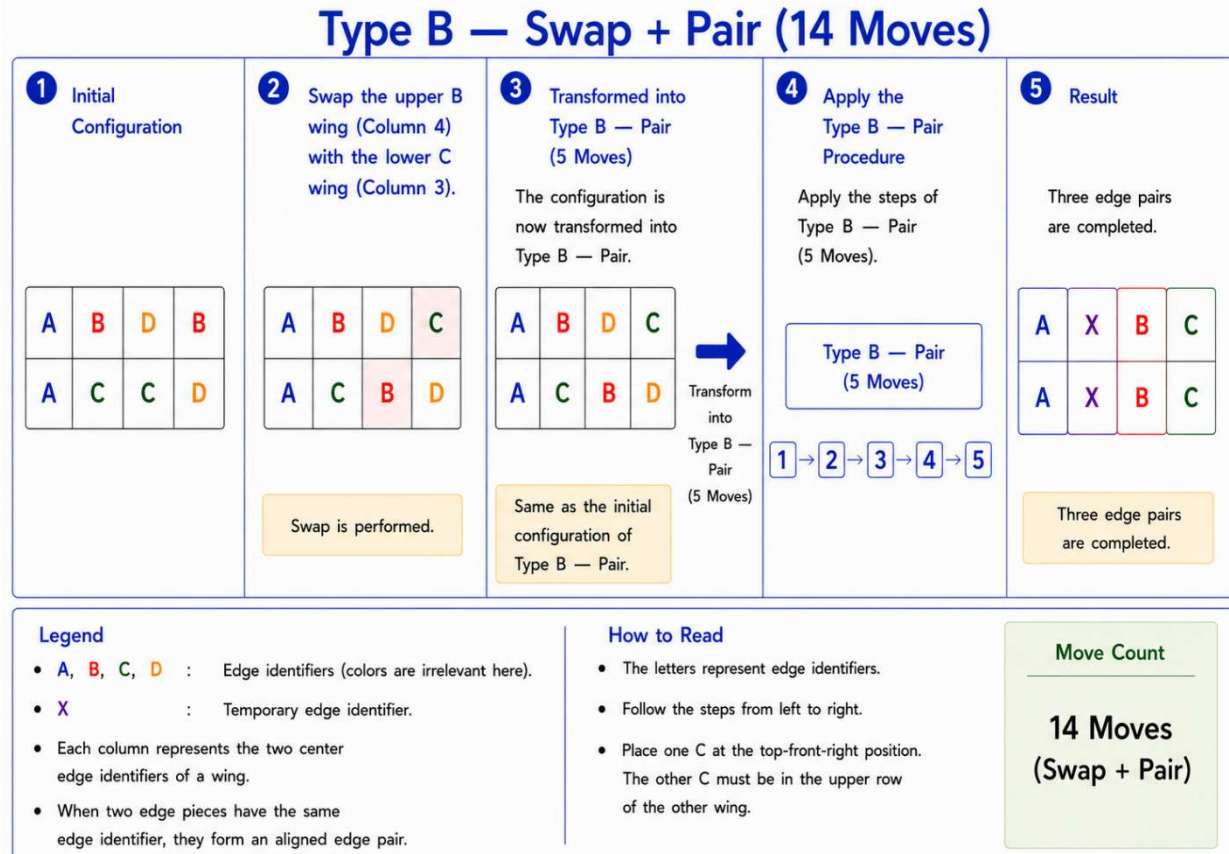


Figure 4.10: Type B — Swap + Pair (14 Moves).

8. Summary

The complete pairing strategy can be summarized as follows.

- Perform **Look Ahead**.
- Try **Search Path 1**.
- If both matching pairs are found, apply **Type A** or **Type B**.
- If unsuccessful, try the complementary **Search Path 2**.
- If both matching pairs are found, apply **Type A** or **Type B**.
- Otherwise, perform a **Flip** or **Swap**, then apply **Type A** or **Type B**.
- Repeat the process until all three remaining edge pairs are completed.

By looking ahead before every pairing, this method preserves the **L3E** structure and avoids unnecessarily inducing an **L2E** configuration.

Only **three move counts** need to be remembered throughout Wang's Simple L3E Pairing Method:

- **5 moves** for direct pairing (**Type A** or **Type B**),
- **12 moves** for **Flip + Pair**, and
- **14 moves** for **Swap + Pair**.

Unlike the traditional Yau L3E pairing approach, this method does **not** require memorizing numerous algorithms or exceptional cases. Instead, it combines two complementary search paths with four simple procedures—**Type A**, **Type B**, **Flip**, and **Swap**—to provide a simple, systematic, and easy-to-learn solution while preserving the principal advantage of the Yau edge pairing method: the solved white cross is maintained, allowing the solver to proceed directly to the 3×3 **CFOP** stage without rebuilding the white cross.

Acknowledgments

The author gratefully acknowledges the Rubik's Cube community for generously sharing educational videos, tutorials, and solving techniques online. Watching many of these videos provided valuable opportunities to observe different solving styles and inspired further exploration of simpler and more systematic approaches to the L3E pairing problem.

Wang's Simple L3E Pairing Method presented in this notebook was independently developed through the author's own analysis, classification, and optimization of L3E configurations.

References

- [1] Feliks Zemdeg, *Yau Method for Solving the 4x4 Cube*, CubeSkills, 2020.
- [2] J Perm, *Learn How to Solve a 4x4 in 10 Minutes (Full Yau Method Tutorial)*, YouTube, January 29, 2018.
- [3] J Perm, *4x4 Yau Method: 3-2-3 Edge Pairing (+ Special Cases)*, YouTube, 2017.
- [4] Noah Richardson, *4×4 Last Edges Pairing (Beginner Friendly) | Final Edge Fix Explained*, YouTube, January 2026.

The Wang Method Series

Notebook #5 (Version 1.0)

Structural Classification and State Transition of the 24 Top-Layer Four-Edge-Pair Patterns

A Structural Framework for Understanding Top-Layer Four-Edge-Pair Patterns

Laung-Terng (LT) Wang

August 1, 2026

1. Introduction

The four top-layer edge pairs can be arranged in 24 (= 4!) distinct clockwise permutations. Although these permutations appear quite different, they exhibit a surprisingly simple structural organization.

In this notebook, the solved configuration **R B O G** is defined as the **Golden Pattern**.

Rather than studying each permutation individually, we first classify all 24 top-layer four-edge-pair configurations according to the number and relative positions of their aligned edge pairs. The resulting classification consists of six structural groups. We then show that every illegal state can always be transformed into a legal state through one of two deterministic transition paths.

This notebook complements the practical solution methods presented in the previous notebooks by providing a unified structural perspective.

2. The 24 Top-Layer Four-Edge-Pair Patterns

The 24 possible clockwise permutations are listed in Table 5.1.

Table 5.1. The 24 Top-Layer Four-Edge-Pair Patterns.

Case	Pattern (Clockwise)
1	R B O G
2	R B G O
3	R O B G
4	R O G B
5	R G B O
6	R G O B
7	B R O G
8	B R G O
9	B O R G
10	B O G R
11	B G R O
12	B G O R
13	O R B G
14	O R G B
15	O B R G
16	O B G R
17	O G R B
18	O G B R
19	G R B O
20	G R O B
21	G B R O
22	G B O R
23	G O R B
24	G O B R

Although these 24 patterns appear unrelated, they can be completely classified into only six structural groups.

3. Structural Classification of All 24 Patterns

Figure 5.1 illustrates the structural classification of all 24 patterns.

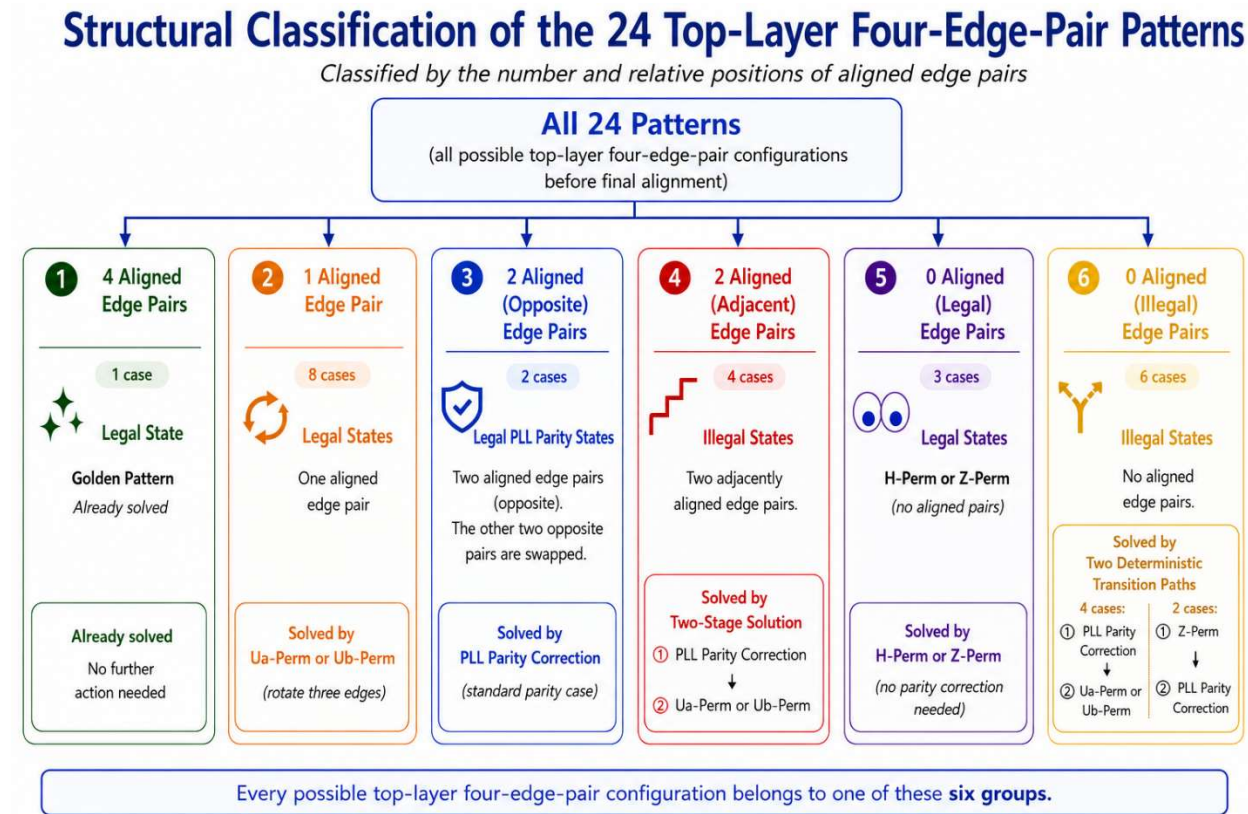


Figure 5.1. Structural classification of the 24 top-layer four-edge-pair patterns before final alignment. The complete classification and corresponding solution methods are summarized in Table 5.2.

Table 5.2. Classification and Solution of All 24 Patterns.

No. of Aligned Edge Pairs	State Type	Case	Pattern (Clockwise)	Solution
4	Golden (Solved)	1	R B O G	Golden Pattern
1	Legal	4	R O G B	<i>Ua-Perm</i>
1	Legal	9	B O R G	<i>Ua-Perm</i>
1	Legal	12	B G O R	<i>Ua-Perm</i>
1	Legal	16	O B G R	<i>Ua-Perm</i>
1	Legal	20	G R O B	<i>Ua-Perm</i>
1	Legal	21	G B R O	<i>Ua-Perm</i>
1	Legal	5	R G B O	<i>Ub-Perm</i>
1	Legal	13	O R B G	<i>Ub-Perm</i>
2	Legal (Parity)	6	R G O B	PLL Parity Correction (BG)
2	Legal (Parity)	15	O B R G	PLL Parity Correction (OR)
2	Illegal	2	R B G O	Parity (BO) $\rightarrow$ R O G B $\rightarrow$ <i>Ua-Perm</i>
2	Illegal	3	R O B G	Parity (OG) $\rightarrow$ R G B O $\rightarrow$ <i>Ua-Perm</i>
2	Illegal	7	B R O G	Parity (RG) $\rightarrow$ B G O R $\rightarrow$ <i>Ua-Perm</i>
2	Illegal	22	G B O R	Parity (BR) $\rightarrow$ G R O B $\rightarrow$ <i>Ua-Perm</i>
0	Legal	17	O G R B	<i>H-Perm</i> (OR, GB)
0	Legal	8	B R G O	<i>Z-Perm</i> (BR, GO)
0	Legal	24	G O B R	<i>Z-Perm</i> (GR, OB)
0	Illegal	11	B G R O	Parity (GO) $\rightarrow$ B O R G $\rightarrow$ <i>Ua-Perm</i>
0	Illegal	14	O R G B	Parity (RB) $\rightarrow$ O B G R $\rightarrow$ <i>Ua-Perm</i>
0	Illegal	18	O G B R	Parity (OB) $\rightarrow$ B G O R $\rightarrow$ <i>Ua-Perm</i>
0	Illegal	23	G O R B	Parity (GR) $\rightarrow$ R O G B $\rightarrow$ <i>Ua-Perm</i>
0	Illegal	10	B O G R	<i>Z-Perm</i> (BR, OG) $\rightarrow$ R G O B $\rightarrow$ Parity (GB)
0	Illegal	19	G R B O	<i>Z-Perm</i> (GR, BO) $\rightarrow$ R G O B $\rightarrow$ Parity (GB)

The 24 patterns are classified into the following six groups.

Group 1 — Golden Pattern

Only one pattern has four aligned edge pairs. This is the solved state and is defined as the Golden Pattern.

Group 2 — One Aligned Edge Pair

Eight patterns have one aligned edge pair. These are all legal states and can be solved directly by either a *Ua-Perm* or a *Ub-Perm*.

Group 3 — Two Opposite Edge Pairs Aligned

Two patterns belong to this group. These are the standard legal PLL parity states and can be solved directly by a standard PLL parity correction.

Group 4 — Two Adjacent Edge Pairs Aligned

Four patterns have two adjacent edge pairs aligned. These are illegal states and require a two-stage solution consisting of a PLL parity correction followed by either a *Ua-Perm* or a *Ub-Perm*.

Group 5 — No Aligned Edge Pairs (Legal States)

Three patterns have no aligned edge pairs but are nevertheless legal states. One pattern is solved by an *H-Perm*, and the other two patterns are solved by a *Z-Perm*.

Group 6 — No Aligned Edge Pairs (Illegal States)

Six patterns have no aligned edge pairs and are illegal states.

Four patterns are transformed into legal Ua or Ub states through a PLL parity correction, whereas the remaining two patterns first require a Z -Perm before reaching legal PLL parity states.

3.1 Key Observations

Several important observations can now be made.

First, the number of aligned edge pairs alone does not determine whether a state is legal.

For example,

- two opposite edge pairs aligned always form legal PLL parity states;
- two adjacent edge pairs aligned always form illegal states;
- no aligned edge pairs may represent either legal or illegal states.

Therefore, both the number and the relative positions of aligned edge pairs are essential for determining legality.

Furthermore, a PLL parity correction may produce either a legal Ua state or a legal Ub state, depending on which opposite edge pair is selected during the parity correction. These two transition paths are completely symmetric.

Together, these observations partition all 24 top-layer four-edge-pair configurations into six structural groups and provide a unified framework for understanding their legality and solution methods.

4. Deterministic State Transition of the 10 Illegal States

Among the 24 patterns, 10 are illegal states.

Fortunately, every illegal state can always be transformed into a legal state through one of two deterministic transition paths. Figure 5.2 summarizes these two transition paths.

Eight illegal states are transformed into legal states through a PLL parity correction. Depending on the selected opposite edge pair during the parity correction, the resulting legal state becomes either a Ua state or a Ub state. The corresponding Ua -Perm or Ub -Perm then returns the cube to the Golden Pattern.

The remaining two illegal states first require a Z -Perm, which converts them into standard legal PLL parity states. A standard PLL parity correction then completes the solution.

Therefore, every illegal state follows one of only two deterministic transition paths.

Most importantly, every illegal state is transformed into a legal state in exactly two stages.

State Transition for All 10 Illegal States



Figure 5.2. State transition of all 10 illegal states

5. Conclusions

This notebook presents a complete structural classification of all 24 top-layer four-edge-pair patterns.

The major findings are summarized below.

1. The 24 patterns are completely classified into six structural groups.
2. The legality of a pattern depends on both the number and the relative positions of its aligned edge pairs.
3. Two opposite edge pairs aligned always form legal states, whereas two adjacent edge pairs aligned always form illegal states. States with no aligned edge pairs may be either legal or illegal.
4. Every illegal state can always be transformed into a legal state in exactly two deterministic stages.
5. A PLL parity correction may produce either a legal Ua state or a legal Ub state, depending on the selected opposite edge pair. The two transition paths are completely symmetric.
6. Together, the structural classification and the state-transition framework provide a complete theoretical understanding of the 24 top-layer four-edge-pair patterns and complement the practical solution methods presented in the previous notebooks.

Although different intermediate legal states (Ua or Ub) may be reached, every illegal state ultimately follows one of two deterministic transition paths back to the Golden Pattern.

The Wang Method Series

Notebook #6 (Version 1.0)

Understanding the Last Two Centers of the 5×5 Rubik's Cube

A Memory-Oriented Framework

Laung-Terng (LT) Wang

August 15, 2026

1. Motivation

Many excellent tutorials explain how to solve the last two centers of the 5×5 Rubik's Cube. These methods are highly effective and are particularly suitable for speedcubers who are willing to memorize many algorithms in order to minimize move count.

This notebook has a different objective.

For many recreational cubers, building the first four centers is relatively straightforward, whereas the last two centers often become the most difficult stage of center solving. The difficulty is not manipulating the cube itself, but recognizing numerous patterns and recalling the appropriate algorithms after weeks or months without practice.

Rather than minimizing the total number of moves, the primary goal of this notebook is to minimize the number of concepts and algorithms that must be remembered.

This notebook therefore develops a **memory-oriented framework** based on understanding, transformation, and pattern reduction. The objective is to replace memorization with logical reasoning whenever possible.

Readers may choose between two different learning styles.

- **Move-oriented solving**, which memorizes more direct algorithms to reduce move count.
- **Memory-oriented solving**, which uses a small number of primitive operations together with simple transformations.

Neither approach is universally better. Competitive speedcubers naturally prefer shorter solutions, whereas recreational cubers and senior hobbyists may find that remembering fewer algorithms is a much more practical long-term strategy.

2. Reducing the Complexity

At first glance, the last two centers appear to be an overwhelming problem.

Each remaining center contains nine pieces. Consequently, the two unsolved centers contain eighteen positions occupied by nine green pieces and nine orange pieces.

Even when pieces of the same color are considered identical, the number of possible color arrangements is

$$\binom{18}{9} = \frac{18!}{9!9!} = 48,620$$

Clearly, no one can recognize or memorize so many configurations.

Fortunately, we do not need to.

The first reduction is to construct one complete three-piece middle line in each center.

After these two middle lines have been completed, six center pieces become fixed, leaving only twelve unresolved positions. The number of possible color arrangements immediately decreases to

$$\binom{12}{6} = \frac{12!}{6! 6!} = 924$$

Although this is a dramatic reduction, 924 configurations are still far too many for practical pattern recognition.

The second reduction is to construct another complete three-piece line, either on the left side or on the right side of each center.

At this stage, each center contains only one unfinished three-piece right line. Altogether, only six center pieces remain unresolved: three green pieces and three orange pieces.

The remaining search space therefore contains only

$$\binom{6}{3} = \frac{6!}{3! 3!} = 20$$

possible color arrangements.

This reduction from

48,620

↓

924

↓

20

forms the central idea of this notebook.

Instead of attempting to recognize tens of thousands of center configurations, we first construct stable three-piece lines that reduce the remaining search space to only twenty reduced states.

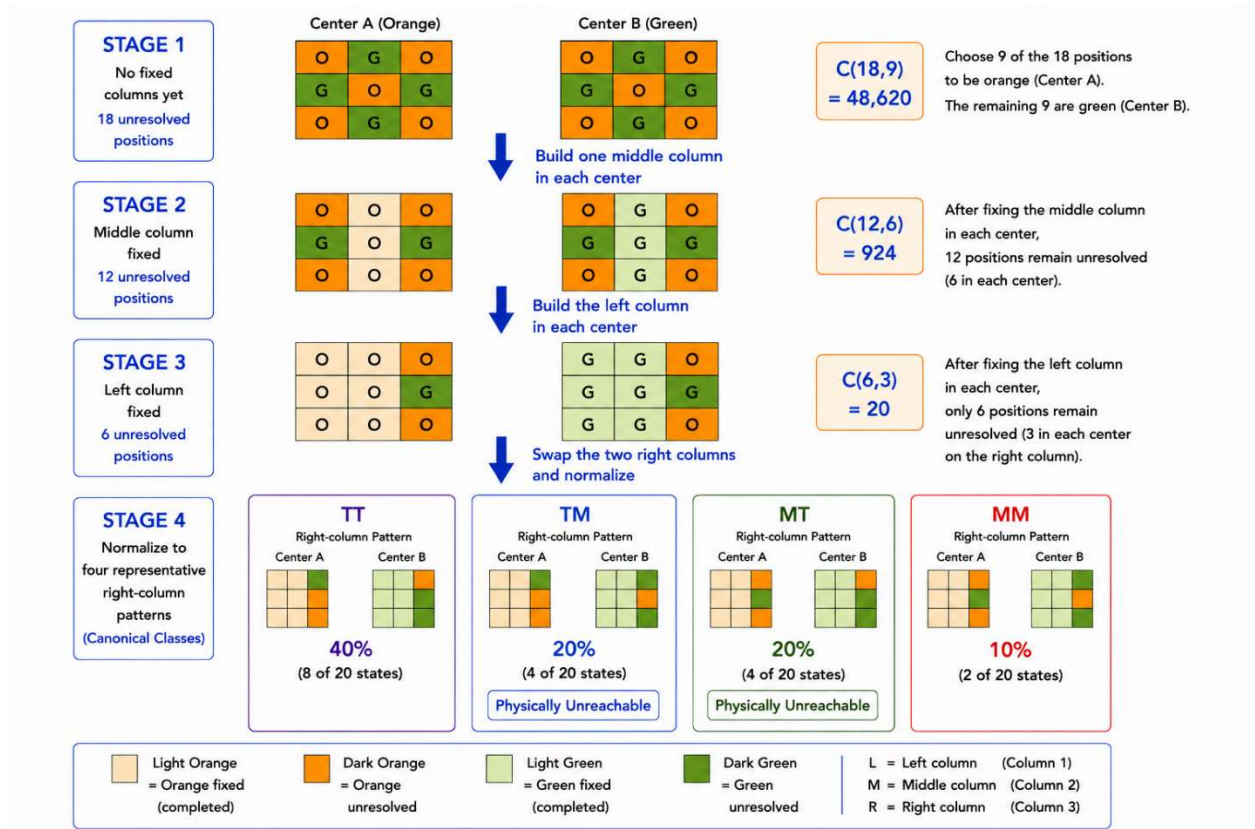


Figure 6.1: Complexity reduction from 48,620 → 924 → 20, together with the combinatorial state distribution.

3. The Twenty Reduced States

Throughout this notebook, the orange center remains on the **top face** and the green center remains on the **front face**.

After two complete three-piece lines have been constructed, only one unfinished right line remains in each center.

Rather than using binary notation, this notebook directly represents the unfinished right lines by their colors:

- **O** denotes an orange center piece.
- **G** denotes a green center piece.

For example,

- **OOO** represents a solved orange right line.
- **GGG** represents a solved green right line.

This notation is considerably more intuitive than binary notation because readers can immediately visualize the remaining center pieces without mentally translating 0s and 1s into colors.

Each unfinished right line contains three positions:

- Top (T)
- Middle (M)
- Bottom (B)

Using this representation, the twenty reduced states obtained after constructing the first two three-piece lines are summarized in Table 6.1.

Table 6.1. The Twenty Reduced Last-Two-Center States.

Case	Original state	Original errors G/O	After right-line swap	Errors after swap G/O	After optional 180° rotation for each B to T	Normalized errors G/O
1	OOO / GGG	— / —	No swap	— / —	No rotation	Solved
2	OOG / OGG	B / T	No swap	B / T	GOO / OGG	T / T
3	OOG / GOG	B / M	No swap	B / M	GOO / GOG	T / M
4	OOG / GGO	B / B	No swap	B / B	GOO / OGG	T / T
5	OGO / OGG	M / T	No swap	M / T	No rotation	M / T
6	OGO / GOG	M / M	No swap	M / M	No rotation	M / M
7	OGO / GGO	M / B	No swap	M / B	OGO / OGG	M / T
8	GOO / OGG	T / T	No swap	T / T	No rotation	T / T
9	GOO / GOG	T / M	No swap	T / M	No rotation	T / M
10	GOO / GGO	T / B	No swap	T / B	GOO / OGG	T / T
11	OGG / OOG	MB / TM	OOG / OGG	B / T	GOO / OGG	T / T
12	OGG / OGO	MB / TB	OGO / OGG	M / T	No rotation	M / T
13	OGG / GOO	MB / MB	GOO / OGG	T / T	No rotation	T / T
14	GOG / OOG	TB / TM	OOG / GOG	B / M	GOO / GOG	T / M
15	GOG / OGO	TB / TB	OGO / GOG	M / M	No rotation	M / M
16	GOG / GOO	TB / MB	GOO / GOG	T / M	No rotation	T / M
17	GGO / OOG	TM / TM	OOG / GGO	B / B	GOO / OGG	T / T
18	GGO / OGO	TM / TB	OGO / GGO	M / B	OGO / OGG	M / T
19	GGO / GOO	TM / MB	GOO / GGO	T / B	GGO / OGG	T / T
20	GGG / OOO	TMB / TMB	OOO / GGG	— / —	No rotation	Solved

Instead of analyzing 48,620 possible center configurations, we now need to understand only twenty reduced states.

These twenty states are not equally important.

After applying right-line swaps and simple cube rotations for normalization, they naturally fall into four canonical unsolved classes together with the solved state.

The occurrence frequencies are summarized in Table 6.2.

Table 6.2. Occurrence frequencies of 20 states.

Class	Number of Cases	Fraction of 20 States
Solved	2	10%
TT	8	40%
TM	4	20%
MT	4	20%
MM	2	10%

Several interesting observations immediately follow.

First, the **TT** family represents 8 of the 20 combinatorial reduced states, or 40% of the abstract state space.

Second, the **MM** family represents 2 of the 20 states, or 10% of the abstract state space.

Finally, although the combinatorial model produces TM and MT configurations, these configurations are not physically reachable on a real 5×5 cube. The reduced color model does not preserve the fixed inventory of physical center pieces. For example, some TM and MT configurations would require a total of three or five corner pieces of one color across the last two centers, whereas a real 5×5 cube has exactly four. Thus, a configuration may be combinatorially valid in the reduced color model without being physically realizable on the cube.

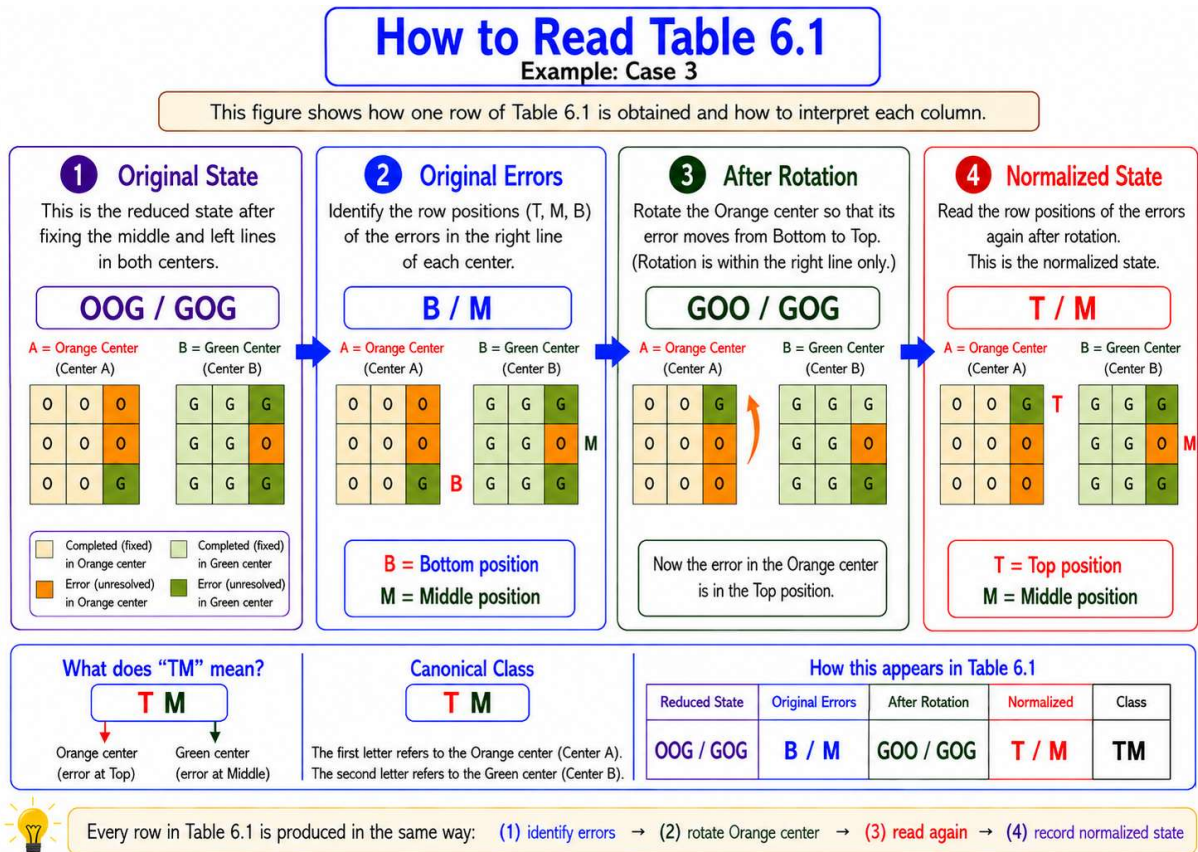


Figure 6.2: How to read Table 6.1 and illustrate the swap and rotation operations.

4. From Twenty Reduced States to Four Canonical Classes

The twenty reduced states are not independent.

The first simplification is obtained by exchanging the two unfinished right lines. This simple operation transforms Cases 11–20 into Cases 1–10. Consequently, only the first ten cases need to be analyzed.

The second simplification is achieved by a simple setup rotation.

Whenever an error appears at the bottom position (B), the corresponding center layer may be rotated so that the bottom error is moved to the top position (T). This normalization does not change the relative relationship between the two unfinished right lines; it merely provides a common reference orientation.

Consequently,

- TT includes TT, TB, BT, and BB.
- TM includes TM and BM.
- MT includes MT and MB.
- MM remains unchanged.

After applying these two normalization procedures, the original twenty reduced states are classified into only four canonical unsolved classes together with the solved state.

The normalization process is illustrated in Figure 6.3, which shows how right-line swaps and simple setup rotations reduce twenty reduced states into only four canonical classes.

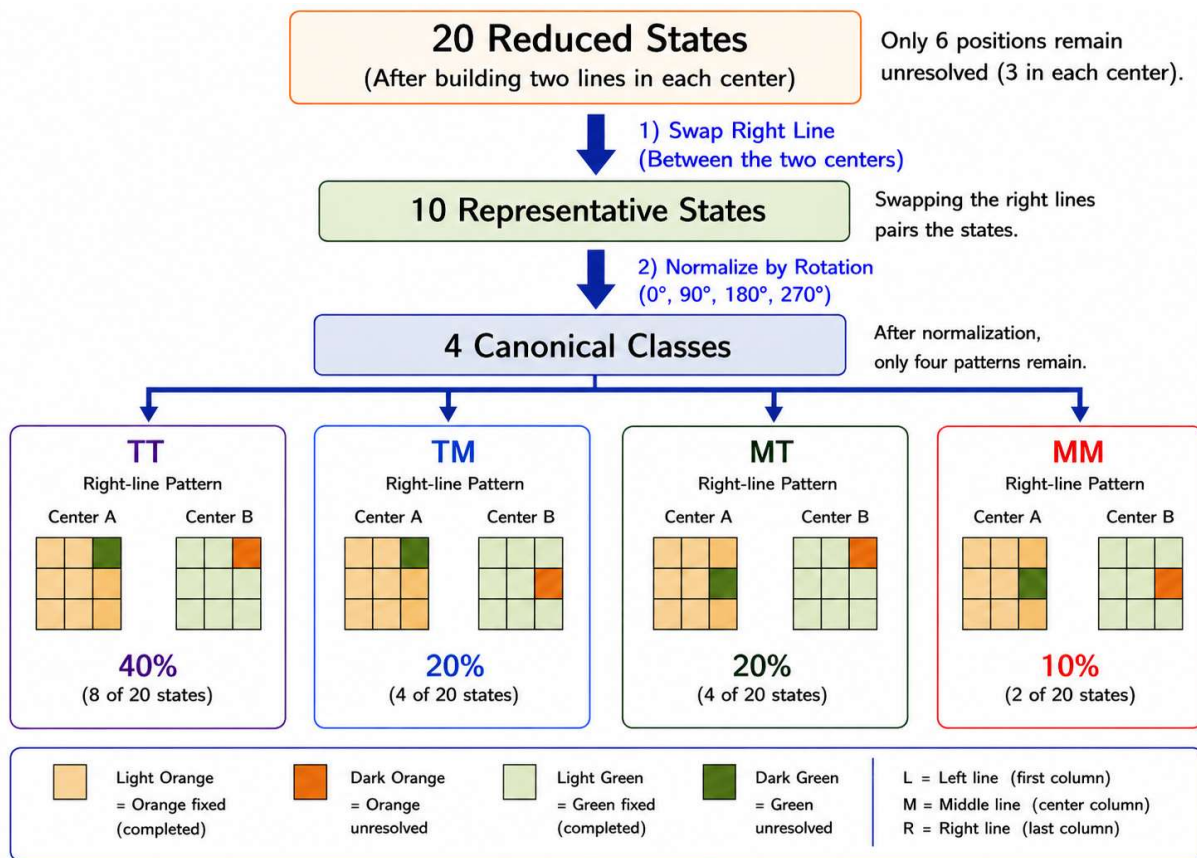


Figure 6.3: Reduction from 20 reduced states to Solved, TT, TM, MT, and MM.

This classification provides a much simpler framework for understanding the last two centers than treating all twenty states independently.

5. Two Alternative Solutions for the TT Family

Among the four canonical classes, the TT family is particularly important because it represents 8 of the 20 combinatorial reduced states.

Two fundamentally different solution strategies are available.

Memory-Oriented Transformation

The first strategy minimizes memorization.

Instead of directly applying a seven-move primitive, the TB case is first transformed by exchanging the two unfinished right lines.

Depending on the orientation, this transformation produces one of two simple local configurations.

Each configuration can then be solved using the familiar four-move local sequence

$U' R w U R w'$

or its equivalent front-face version

$F' R w' F R w$.

Although this approach requires approximately fourteen moves in total, every step is local, intuitive, and easy to reconstruct after long periods without practice.

The complete procedure is illustrated in Figure 6.4.

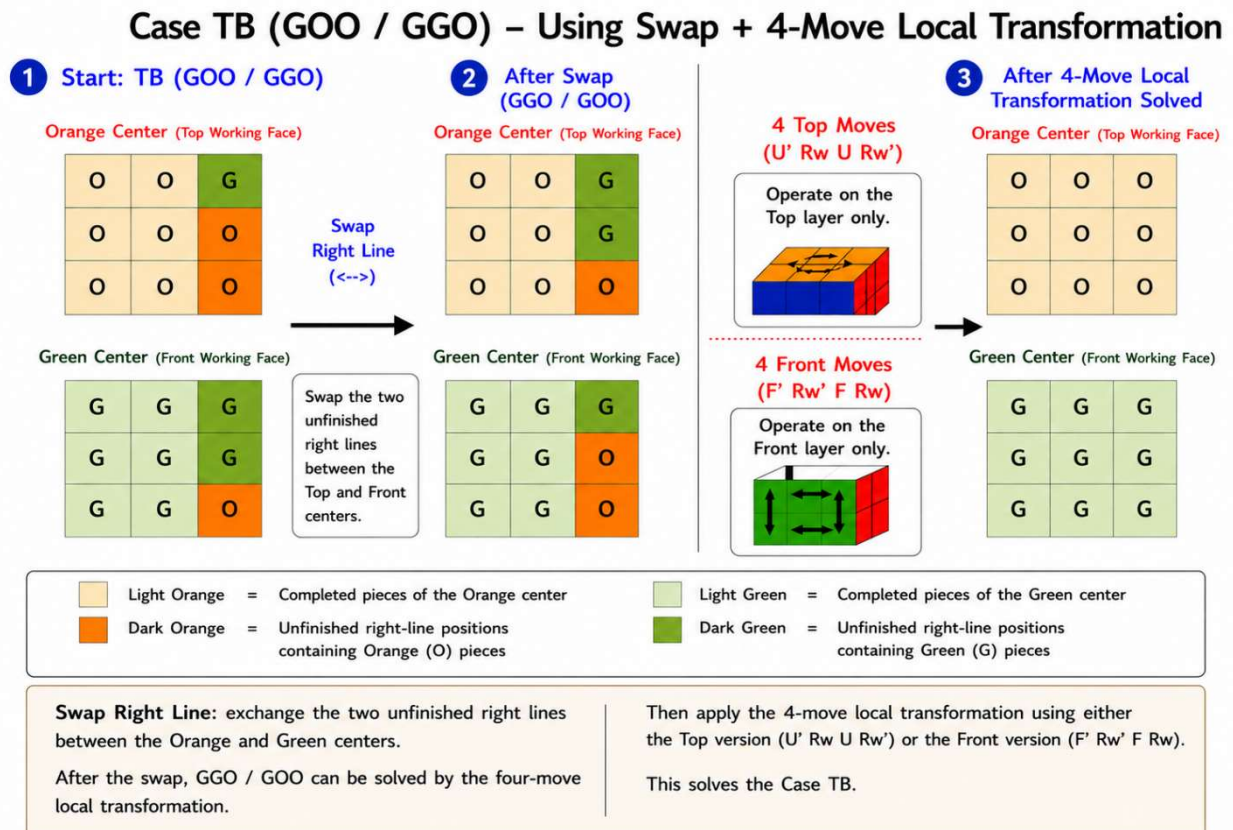


Figure 6.4: Using swap and 4-move local transformation to solve TB (GOO/GGO).

Direct TT Primitive

The second strategy minimizes move count. **The direct TT and MM move sequences are commonly referred to as commutators in the cubing community.**

With the orange center on the top face and the green center on the front face, one representative TT primitive is

Rw' F' Lw F Rw F' Lw'

requiring approximately seven moves.

Unlike the previous method, this primitive temporarily borrows workspace from a neighboring completed center before restoring it at the end of the sequence.

Rather than viewing this simply as a memorized algorithm, it is helpful to understand its three functional stages:

1. Borrow temporary workspace.
2. Transfer the required center pieces.
3. Restore the borrowed workspace.

Understanding these functions makes the primitive easier to reconstruct, even if the exact move sequence is temporarily forgotten.

The two approaches therefore represent different design philosophies.

The direct primitive minimizes move count. The transformation method minimizes memory burden. Readers may choose whichever approach best matches their own learning style.

As another practical option, cubers may transform a TT state to BT and then apply the right Sune algorithm. This may be especially convenient for cubers who are already familiar with the right Sune for solving OLL corners during 4×4 CFOP.

More generally, cubers do not need to memorize a separate direct algorithm for every case. An unfamiliar case may instead be transformed into a familiar case whose algorithm is already known. The preferred transformation is therefore not necessarily the one with the fewest setup moves, but one that provides a favorable tradeoff between move count and memory burden. In some cases, a transformation may add only one or a few moves while eliminating the need to memorize an additional algorithm.

For example, the TB/TB case can be transformed to MM in four moves and then solved with the seven-move MM algorithm, for a total of 11 moves—the same move count as its direct solution. Thus, cubers who already know the MM algorithm need not memorize a separate TB/TB algorithm.

6. Why TM and MT Do Not Occur Physically

The combinatorial reduction generates four TM states and four MT states among the twenty reduced color arrangements. However, these eight states are not physically reachable on a real 5×5 cube.

The reason is that the reduced model tracks only the colors appearing in the two remaining centers; it does not preserve the fixed inventory of the physical center pieces. For example, some of the abstract TM and MT arrangements would require a total of three or five corner pieces of one color across the last two centers, rather than the four physically available on the cube. Such arrangements can therefore exist in the combinatorial model but cannot occur as legal physical states.

This explains why TM and MT appear naturally in the twenty-state combinatorial classification even though they are not encountered when solving a real cube. It also illustrates an important distinction between a **combinatorially valid color arrangement** and a **physically reachable cube state**.

7. Two Learning Paths

Throughout this investigation, two complementary approaches for solving the last two centers have emerged.

The first is a **move-oriented approach**, which minimizes the total number of moves by memorizing a larger collection of direct algorithms.

The second is the **memory-oriented approach** developed in this notebook, which minimizes the number of algorithms by using simple transformations together with a small number of primitive operations.

These two approaches represent different learning philosophies rather than competing methods.

For speedcubers who solve the cube every day, memorizing additional algorithms may be entirely reasonable if it reduces move count.

For recreational cubers, however, the situation is often quite different.

After already learning OLL, PLL, and parity algorithms, many hobbyists may find it difficult to memorize another large collection of L2C algorithms and retain them after weeks or months without practice.

The memory-oriented framework therefore seeks a different balance.

Instead of asking,

"Which algorithm should I memorize?"

it asks,

"Can this configuration first be transformed into one that I already understand?"

This philosophy trades a modest increase in move count for a substantial reduction in memory burden. Readers may therefore choose whichever approach best matches their own goals and learning preferences.

For convenient use during practice, the complete set of four L2C reference cards is provided in the Appendix.

8. Lessons Learned

This investigation began with a simple question:

Why are the last two centers so difficult?

At first glance, the answer appeared to be the large number of possible patterns and the many algorithms required to solve them.

The analysis presented in this notebook suggests a different explanation. The true difficulty lies not in the enormous number of theoretical configurations, but in the limited temporary workspace available after most center pieces have already been completed.

By progressively constructing stable three-piece lines, the search space is reduced from

$$\binom{18}{9} = \frac{18!}{9! 9!} = 48,620$$

to

$$\binom{12}{6} = \frac{12!}{6! 6!} = 924$$

and finally to

$$\binom{6}{3} = \frac{6!}{3! 3!} = 20$$

The remaining twenty reduced states can then be understood systematically through swap and rotation, leading to only a small number of canonical solution families. More importantly, this study illustrates a broader principle. A seemingly complicated problem often becomes much simpler after identifying the right representation.

Rather than relying primarily on memorization, one may instead search for transformations that convert unfamiliar situations into familiar ones. This viewpoint forms the central philosophy of the memory-oriented framework presented in this notebook.

9. Concluding Remarks

This notebook does not attempt to present the fastest solution for the last two centers of the 5×5 Rubik's Cube. Instead, it proposes a different way of thinking about the problem.

Its principal contribution is not a new algorithm, but a systematic framework that reduces complexity through mathematical analysis, simple transformations, and a small number of primitive operations.

Readers who prefer the shortest solutions may choose to memorize a larger collection of direct algorithms. Readers who prefer a lighter memory burden may instead adopt the memory-oriented approach developed here.

Both approaches are valid. They simply optimize different objectives.

Acknowledgments

The author gratefully acknowledges Marty Kelsey (Martyt7777) for his valuable insights and helpful discussions on center-solving algorithms, which helped inspire the systematic study of the 5×5 last-two-center cases presented in this notebook. The author also thanks members of the SpeedSolving.com community who generously shared references, links, technical comments, and their cubing experience during the development of this work.

Appendix. L2C Reference Cards

The following four reference cards summarize the last two centers (L2C) cases and algorithms discussed in this notebook. They are included as a practical companion to the combinatorial classification developed in the main text and are intended for quick reference during solving and practice.

Reference Cards – Representative Cases

Direct Solutions for the Last Two Centers (L2C)

CASE 1

SOLVED
(OOO / GGG)

Orange Center
(Top Working Face)

O	O	O
O	O	O
O	O	O

Green Center
(Front Working Face)

G	G	G
G	G	G
G	G	G

Direct Solution

Already Solved
(No moves needed)

CASE 8

TT
(GOO / OGG)

Orange Center
(Top Working Face)

O	O	G
O	O	O
O	O	O

Green Center
(Front Working Face)

G	G	O
G	G	G
G	G	G

Direct Solution

Rw' F' Lw F Rw F' Lw'
(7 moves)

CASE 6

MM
(OGO / GOG)

Orange Center
(Top Working Face)

O	O	O
O	O	G
O	O	O

Green Center
(Front Working Face)

G	G	G
G	G	O
G	G	G

Direct Solution

Rw U' M' U Rw' U' M
(7 moves)

= Correctly placed Orange center pieces

= Misplaced (unfinished) positions containing Orange (O) pieces

= Correctly placed Green center pieces

= Misplaced (unfinished) positions containing Green (G) pieces

Note: T = Top; M = Middle; B = Bottom

Figure 6.5: Possible algorithms to solve Cases 1, 8, and 6.

Reference Cards – Representative Cases

Direct Solutions for the Last Two Centers (L2C)

CASE 2

BT
(OOG / OGG)

Orange Center
(Top Working Face)

O	O	O
O	O	O
O	O	G

Green Center
(Front Working Face)

G	G	O
G	G	G
G	G	G

Direct Solution

(Rw U Rw') U (Rw U2' Rw')

CASE 4

BB
(OOG / GGO)

Orange Center
(Top Working Face)

O	O	O
O	O	O
O	O	G

Green Center
(Front Working Face)

G	G	G
G	G	G
G	G	O

Direct Solution

Rw' F Lw F' Rw F Lw'

CASE 10

TB
(GOO / GGO)

Orange Center
(Top Working Face)

O	O	G
O	O	O
O	O	O

Green Center
(Front Working Face)

G	G	G
G	G	G
G	G	O

Direct Solution

(Rw U' Rw') U' (Rw U2' Rw')

= Correctly placed Orange center pieces

= Misplaced (unfinished) positions containing Orange (O) pieces

= Correctly placed Green center pieces

= Misplaced (unfinished) positions containing Green (G) pieces

Note: T = Top; M = Middle; B = Bottom

Figure 6.6: Possible algorithms to solve Cases 2, 4, and 10.

Reference Cards – Representative Cases

Direct Solutions for the Last Two Centers (L2C)

CASE 11

MB / TM → TT
(OGG / OOG)

Orange Center
(Top Working Face)

O	O	O
O	O	G
O	O	G

Green Center
(Front Working Face)

G	G	O
G	G	O
G	G	G

Direct Solution

U (Rw U' Rw')

CASE 13

MB / MB → TT
(OGG / GOO)

Orange Center
(Top Working Face)

O	O	O
O	O	G
O	O	G

Green Center
(Front Working Face)

G	G	G
G	G	O
G	G	O

Direct Solution

U (Rw U Rw') U' (Rw U' Rw')

CASE 17

TM / TM → TT
(GGO / OOG)

Orange Center
(Top Working Face)

O	O	G
O	O	G
O	O	O

Green Center
(Front Working Face)

G	G	O
G	G	O
G	G	G

Direct Solution

U' (Rw U' Rw') U (Rw U Rw')

= Correctly placed Orange center pieces

= Misplaced (unfinished) positions containing Orange (O) pieces

= Correctly placed Green center pieces

= Misplaced (unfinished) positions containing Green (G) pieces

Note: T = Top; M = Middle; B = Bottom

Figure 6.7: Possible algorithms to solve Cases 11, 13, and 17.

Reference Cards – Representative Cases

Direct Solutions for the Last Two Centers (L2C)

CASE 19

TM / MB → TT
(GGO / GOO)

Orange Center
(Top Working Face)

O	O	G
O	O	G
O	O	O

Green Center
(Front Working Face)

G	G	G
G	G	O
G	G	O

Direct Solution

U' (Rw U Rw')

CASE 15

TB / TB → MM
(GOG / OGO)

Orange Center
(Top Working Face)

O	O	G
O	O	O
O	O	G

Green Center
(Front Working Face)

G	G	O
G	G	G
G	G	O

Direct Solution

(Rw U' Rw' U) (Rw U' Rw' U) (Rw U' Rw')

CASE 20

TMB / TMB → SOLVED
(GGG / OOO)

Orange Center
(Top Working Face)

O	O	G
O	O	G
O	O	G

Green Center
(Front Working Face)

G	G	O
G	G	O
G	G	O

Direct Solution

U2 (Rw U2 Rw')

= Correctly placed Orange center pieces

= Misplaced (unfinished) positions containing Orange (O) pieces

= Correctly placed Green center pieces

= Misplaced (unfinished) positions containing Green (G) pieces

Note: T = Top; M = Middle; B = Bottom

Figure 6.8: Possible algorithms to solve Cases 19, 15, and 20.

The Wang Method Series
Notebook #7 (Version 1.0)
Comparative Study and Educational Perspectives
Laung-Terng (LT) Wang
August 15, 2026

1. Motivation

The last two centers (L2C) of the 5×5 cube can be solved using different strategies. Rather than viewing them as competing methods, they should be regarded as different optimizations for different types of cubers.

This section compares three representative approaches:

1. Noah Richardson's Beginner Method
2. CubeSkills Method
3. The Method Presented in This Notebook

Each method emphasizes a different design objective. Figure 7.1 illustrates the different L2C structures constructed by the three methods: a cross on each of the top and front faces, two vertical columns on the front face, and two vertical columns on each of the top and front faces.

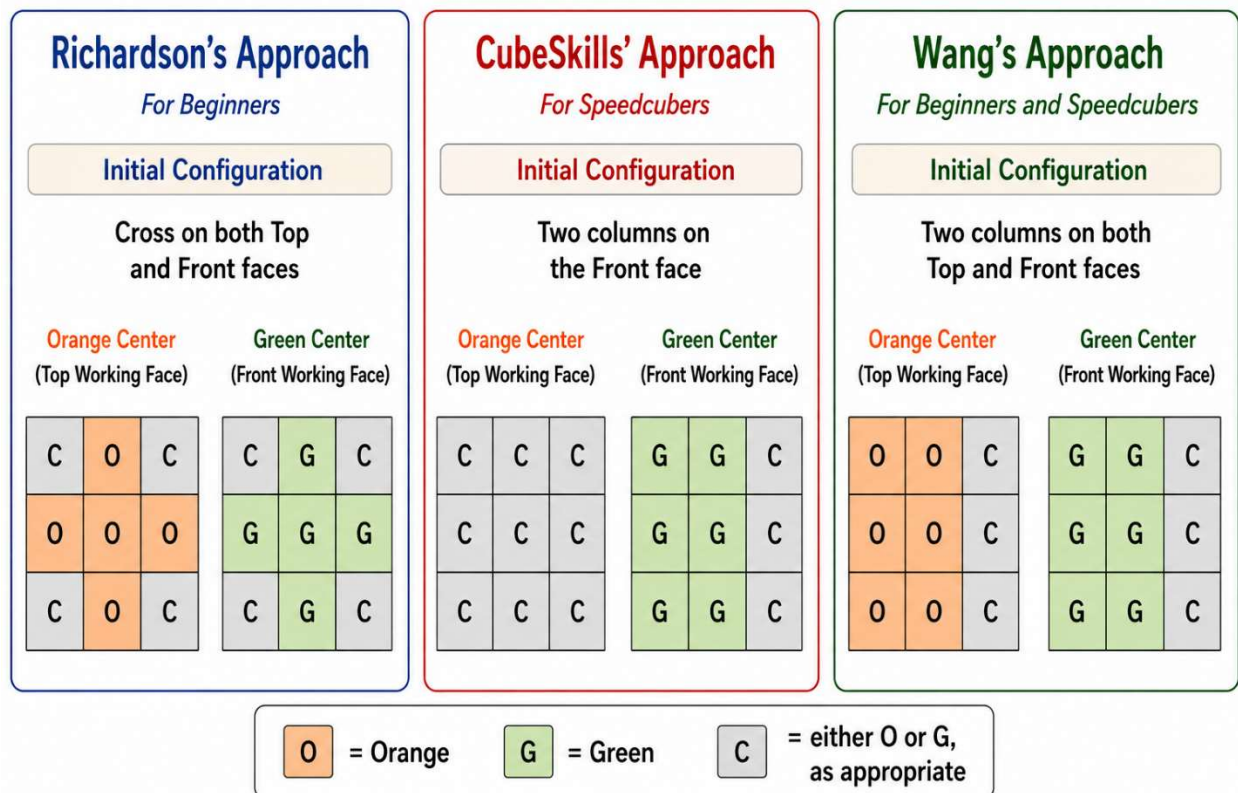


Figure 7.1. Initial configurations used by three common approaches for building the first structures when solving the last two centers on the 5x5 Rubik's Cube.

2. Noah Richardson's Beginner Method

Noah Richardson's method is specifically designed for beginners [1]. Rather than recognizing many different L2C cases and memorizing numerous algorithms, the method follows a fixed construction procedure.

First, a cross is built on one center. A cross is then built on the opposite center. After the two crosses have been completed, only the four corner positions on each center remain unsolved.

Unlike methods that recognize many representative patterns, Noah's method repeatedly applies a single primitive algorithm (Right Sune), together with simple cube rotations, until all remaining corner pieces are solved. This greatly reduces the memorization burden and makes the method particularly attractive for beginning cubers.

Remaining Corner Configurations

After both crosses have been completed, only the corner pieces remain unsolved.

There are

- four corner positions on the orange center, and
- four corner positions on the green center.

Therefore, eight corner positions remain.

Among these eight positions are exactly

- four orange center pieces, and
- four green center pieces.

Since pieces of the same color are indistinguishable, a complete corner configuration is determined simply by selecting which four of the eight positions contain the orange pieces. The remaining four positions must then contain the green pieces.

Hence, the total number of possible corner configurations is

$$\binom{8}{4} = \frac{8!}{4! 4!} = 70$$

This simple combinatorial argument provides an intuitive explanation for why exactly 70 corner configurations are possible after the two crosses have been completed.

Mismatched Corner Pairs

Suppose exactly k orange pieces are located on the green center.

Since the numbers of orange and green pieces are equal, there must also be exactly k green pieces located on the orange center.

Each misplaced orange piece together with its corresponding misplaced green piece forms one **mismatched corner pair**.

To obtain exactly k mismatched corner pairs,

- choose k of the four green-center corners to contain orange pieces, and
- choose k of the four orange-center corners to contain green pieces.

Therefore,

$$N(k) = \binom{4}{k} \binom{4}{k} = \binom{4}{k}^2$$

and the corresponding probability is

$$P(k) = \frac{\binom{4}{k}^2}{\binom{8}{4}} = \frac{\binom{4}{k}^2}{70}$$

Solving the Remaining Corner Pairs

The key idea of Noah Richardson's beginner method is remarkably simple.

One application of the 7-move Right Sune algorithm, together with appropriate setup rotations, swaps and corrects **one mismatched corner pair**.

Consequently,

- one mismatched pair requires one Right Sune;
- two mismatched pairs require two Right Sunes;
- three mismatched pairs require three Right Sunes;
- four mismatched pairs require four Right Sunes.

Therefore, if k mismatched corner pairs remain,

$$M(k) = 7k$$

where $M(k)$ denotes the required move count after the two crosses have been completed.

The complete probability distribution is summarized in Table 7.1.

Table 7.1. Probability distribution of Noah Richardson's Beginner Method.

Mismatched Corner Pairs	Configurations	State Distribution	Right Sune Applications	Moves
0	1	1.43%	0	0
1	16	22.86%	1	7
2	36	51.43%	2	14
3	16	22.86%	3	21
4	1	1.43%	4	28
Total	70	100%	—	—

The distribution

$$1 : 16 : 36 : 16 : 1$$

is perfectly symmetric. The most common situation consists of two mismatched corner pairs, accounting for over half of all possible configurations.

Average Move Count

The theoretical average move count is obtained by taking the probability-weighted average:

$$E[M] = \frac{1(0) + 16(7) + 36(14) + 16(21) + 1(28)}{70} = \frac{980}{70} = 14$$

Equivalently, the expected number of mismatched corner pairs is

$$E[k] = \frac{0(1) + 1(16) + 2(36) + 3(16) + 4(1)}{70} = \frac{140}{70} = 2$$

and since each mismatched corner pair requires one 7-move Right Sune,

$$E[M] = 7E[k] = 7 \times 2 = 14$$

Thus, after the two crosses have been completed, Noah Richardson's beginner method requires an average of **14 moves** to solve the remaining corner pairs.

The principal strength of Noah Richardson's method lies in its simplicity rather than move efficiency.

By repeatedly applying the same primitive algorithm, beginners need to memorize only one core transformation while following a fixed construction strategy. Although the move count may vary from 0 to 28 moves depending on the number of mismatched corner pairs, the theoretical average is exactly **14 moves**.

This elegant trade-off between memorization and move count makes the method an excellent educational approach for beginning cubers.

3. CubeSkills Method

The CubeSkills approach targets speedcubers rather than beginners [2]. Before applying its L2C algorithms, the solver first constructs a solved 2×3 center block on the front face.

CubeSkills states:

These L2C algorithms are for the cases where a 2×3 block of center pieces is solved on the front face. After this construction, CubeSkills presents **20 representative patterns**, each associated with a direct algorithm.

The listed solutions typically require approximately

- 3 moves,
- 7 moves,
- or 11 moves,

depending on the configuration.

To assist memorization, CubeSkills places parentheses around portions of longer algorithms, grouping familiar move triggers into smaller units.

This organization is particularly useful for experienced speedcubers who are already accustomed to memorizing large collections of algorithms such as one-look OLL and one-look PLL.

For such cubers, memorizing an additional set of L2C algorithms may represent a reasonable trade-off for shorter solutions.

Completeness

CubeSkills presents 20 practical patterns.

However, the reference does not appear to provide a mathematical proof that these 20 patterns constitute the complete set of all reachable L2C configurations after the prescribed 2×3 construction.

Consequently, the 20 CubeSkills patterns should be viewed as an effective speed-solving algorithm library rather than a mathematically derived classification of the complete state space.

4. The Method Presented in This Notebook

Unlike the previous two approaches, this notebook emphasizes understanding of the underlying state space [3]. The analysis begins by reducing the remaining configurations into canonical representative states through symmetry, rotations, and swap operations.

This framework naturally supports two operating modes.

Beginner Mode

The beginner mode minimizes memorization. Representative cases are normalized by swap and rotation so that only TT and MM algorithms are required.

The resulting solutions typically require approximately 11-12 moves, while requiring only a very small algorithm library. This mode is intended for casual cubers who solve cubes only occasionally.

Direct Mode

The direct mode eliminates normalization. Instead, each representative case is solved directly using its own algorithm.

For the representative cases currently identified, the solution lengths are approximately

- 4 moves,
- 8 moves,
- 11 moves,

including one setup rotation when required.

Compared with the beginner mode, this approach requires memorizing additional representative algorithms but consistently produces substantially shorter solutions. Twelve representative algorithms are sufficient.

5. Overall Comparison

Table 7.2 compares key features of the three methods. Beginners and speedcubers can therefore choose the approach best suited to their needs.

Table 7.2. Comparison of key features of three L2C solving approaches.

Feature	Noah	CubeSkills	This Notebook
Primary Objective	Minimize memorization	Minimize move count	Balance memory and move count
Target Users	Beginners	Speedcubers	Beginners to advanced cubers
Initial Construction	Two crosses	2×3 front block	Canonical-state reduction
Representative Cases	Repeated primitive algorithm	20 listed patterns	Canonical representative states
Algorithms	One core algorithm	About 20 direct algorithms	TT/MM mode or Direct mode
Typical Moves	Average 14	3–7–11	Beginner: 11–12; Direct: 4–8–11
Main Strength	Simplicity	Speed	Flexibility and state-space understanding

A More Flexible Approach

In practice, cubers do not necessarily need to follow any one of the three approaches described above. Another simple strategy is to place as many center pieces as possible on their correct Top and Front faces without following a predetermined construction pattern. Once only a few opposite-color pieces remain on the two centers, they can be solved one pair at a time using familiar TT or MM commutators. This provides a flexible alternative for cubers who prefer intuitive piece placement over a fixed center-building structure.

6. Discussion

The three methods presented in this chapter should not be viewed as competing approaches. Instead, they represent three different design philosophies for solving the last two centers (L2C), each optimizing a different objective.

Noah Richardson's method simplifies the **construction process**. By building two crosses first, the remaining configurations are greatly simplified and can be solved by repeatedly applying a single primitive

algorithm (Right Sune). This elegant construction-based strategy minimizes algorithm memorization and provides an excellent learning path for beginners.

The CubeSkills method simplifies the **solution length**. After constructing a 2×3 center block, it recognizes representative patterns and applies direct algorithms, typically requiring only 3, 7, or 11 moves. This approach is well suited for competitive speedcubers who are willing to memorize a larger collection of algorithms. However, the CubeSkills reference presents 20 practical patterns without establishing mathematically whether they constitute a complete classification of all reachable L2C configurations under its construction.

The method presented in this notebook simplifies the **state-space analysis**. Through canonical-state reduction, symmetry, rotations, and swap operations, the remaining configurations are organized into a systematic analytical framework. This framework supports two operating modes. The beginner mode minimizes memorization through normalization, whereas the direct mode trades additional memorization for consistently shorter solutions.

Readers may therefore select the approach that best matches their own objectives:

- **Beginners** may prefer Noah Richardson's simple construction method.
- **Casual cubers** may prefer the notebook's normalized TT/MM beginner mode with its small algorithm library.
- **Intermediate and advanced cubers** may choose the notebook's direct representative algorithms for shorter solutions.
- **Competitive speedcubers** may adopt larger algorithm libraries such as CubeSkills to minimize move count.

Ultimately, these three methods complement one another rather than compete with one another. Together they illustrate three different perspectives on solving the last two centers:

- **Noah Richardson's method** emphasizes construction.
- **CubeSkills** emphasizes move efficiency.
- **This notebook** emphasizes mathematical understanding and state-space analysis.

Each perspective offers valuable insights into the L2C problem. Taken together, they demonstrate how different optimization objectives—construction strategy, algorithm memorization, move efficiency, and mathematical analysis—can lead to different, yet equally elegant, solutions.

References

- [1] Noah Richardson, “ 5×5 LAST 2 CENTERS | EASY TUTORIAL,” YouTube.
<https://www.youtube.com/watch?v=Vv5j1Tyz6Ew>
- [2] CubeSkills, “ 5×5 L2C Algorithms (Last Two Centers),” *Last 2 Centers Algorithms [5x5]*.
<https://cubeskills.com/uploads/pdf/tutorials/last-2-centers-algorithms-5x5.pdf>
- [3] L.-T. Wang, *Understanding the Last Two Centers of the 5×5 Rubik's Cube: A Memory-Oriented Framework*, The Wang Method Series, Notebook #6, Version 1.0, Aug. 15, 2026.
https://ltwang888.github.io/resources/LT_Wang_Notebook_6_August_15_2026_v1.0.pdf

The Wang Method Series

Notebook #8 (Version 1.0)

A Minimal-Memory Approach to 6×6 Last Two Centers (L2C)

A Beginner-Friendly Reference Framework Using Commutator-Based Local Swaps

Laung-Terng (LT) Wang

August 18, 2026

1. Introduction

The Last Two Centers (L2C) step of a 6×6 cube appears significantly more complicated than that of smaller cubes because each center consists of a 4×4 block. Some approaches require memorizing many case-specific algorithms, which may create a substantial memory burden for beginners.

For a casual beginner, however, the primary challenge is not move optimization but algorithmic memory. After learning CFOP and solving 4×4 and 5×5 cubes, adding a large collection of additional 6×6 L2C algorithms can become overwhelming.

This note presents a minimal-memory approach for 6×6 L2C. Instead of memorizing numerous individual cases, the beginner first reduces the configuration intuitively whenever possible and then applies a small set of local commutator-based operations to solve the remaining pieces.

In the cubing community, **commutators** refer to sequences designed to swap a small number of target pieces while preserving the rest of the cube. This locality property is especially valuable for L2C solving because previously completed center colors should not be disturbed.

Experienced cubers often recognize that apparently different cases may share similar underlying structures through commutators, conjugates, and setup transformations. This note adopts a beginner-oriented perspective by focusing on visual recognition, local swaps, and memory reduction.

For 6×6 L2C, the remaining configurations can often be addressed using three types of commutator-based local swaps:

- **TT (Top–Top) swap:** a local swap involving the topmost target center positions on the working faces.
- **MM (Middle–Middle) swap:** a local swap involving the middle edge-pair target positions on the working faces.
- **CC (Center–Center) swap:** a local swap involving the inner 2×2 center block positions on the working faces.

The detailed definitions and representative configurations of TT, MM, and CC structures are provided in Section 3.

The goal of this approach is not speed solving. Instead, it is to provide a beginner-friendly framework that minimizes memorization requirements while maintaining a reliable solving process.

2. Reduction Strategy

A 6×6 center consists of a 4×4 block. During L2C solving, pieces that can be conveniently moved to their correct locations are handled first.

A beginner does not need to construct every center position through predefined algorithms. Instead, the configuration can first be reduced intuitively by placing as many center pieces as possible into their correct locations.

After intuitive reduction and swapping, the remaining pieces may not always form perfectly separated edge pairs. Adjacent or partially connected center pieces may remain. These residual configurations do not necessarily require completely new algorithms.

Instead, the cuber can identify the remaining local structure and apply an appropriate commutator-based repair operation.

The guiding principle is:

Reduce the configuration whenever convenient, identify the remaining local structure, and repair the pieces using a few reliable commutator-based operations.

This approach avoids the need to memorize a large database of individual L2C algorithms.

3. TT, MM, and CC Commutator Reference Cards

Definition of TT, MM, and CC Structures

In this note, **TT**, **MM**, and **CC** describe the relative locations of the target center pieces involved in local commutator-based swaps.

Using the 4×4 center face representation shown in Figure 8.1:

- **TT (Top–Top swap)** refers to swapping the target center pieces located at the topmost right corner positions within the 4×4 center blocks on the Orange top working face and the Green front working face.
- **MM (Middle–Middle swap)** refers to swapping the middle edge-pair positions. For the right orientation, the target pieces are located at the second rightmost edge-pair positions on the top slice of the Orange top working face and the Green front working face. For the left orientation, the corresponding second leftmost edge-pair positions are used.
- **CC (Center–Center swap)** refers to swapping positions within the inner 2×2 center block located at the center of the 4×4 center area. For the right orientation, the target pieces are located in the right-side positions of the inner 2×2 center block on the Orange top working face and the Green front working face. For the left orientation, the corresponding left-side positions are used.

These names describe the geometric locations of the target pieces rather than complete cube cases. Different residual configurations may be transformed into these local swap structures through rotations, piece rearrangements, or other setup operations.

Figure 8.1 provides six representative reference cards covering the three commutator families used in this approach:

- TT and BT orientations;
- MM right and left orientations;
- CC right and left orientations.

A consistent reference orientation is used throughout the figure:

- The Orange center is shown as the top working face.
- The Green center is shown as the front working face.

Each reference card contains:

1. The representative 4×4 center configuration.
2. The corresponding direct solution sequence.

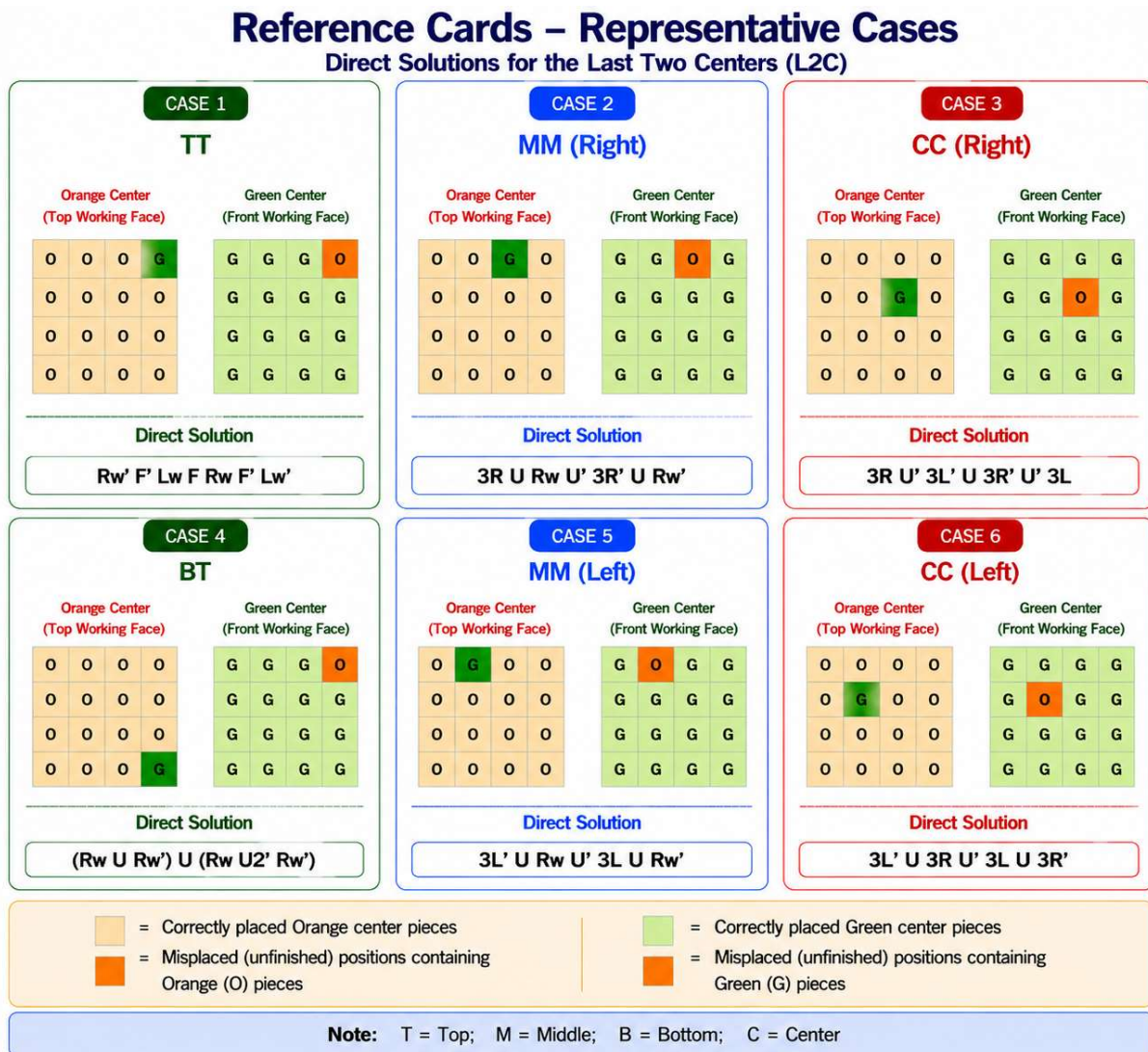


Figure 8.1. Six representative commutator reference cards for 6×6 L2C.

These cards are not intended to represent every possible L2C case. Instead, they provide visual references for recognizing common local structures that frequently appear after intuitive reduction.

A beginner can therefore focus on understanding the relationship between remaining pieces rather than memorizing a large collection of independent algorithms.

The six reference cards represent three commutator families: TT, MM, and CC. Each commutator performs a local swap of target center pieces while preserving the colors of other completed center pieces.

The TT case can either be solved using its direct commutator or transformed into the BT orientation and solved using the familiar Sune operation. MM and CC cases are handled using their corresponding orientation-specific commutators.

4. Example: Solving an MC/MC Residual Configuration

In this note, **MC/MC (Middle–Center/Middle–Center)** refers to an adjacent Middle–Center residual configuration in which one MC structure appears on the Orange top working face and the corresponding MC structure appears on the Green front working face.

Unlike TT, MM, and CC, MC/MC is not a separate commutator family. Instead, it demonstrates how a more complicated residual configuration can be decomposed into familiar local swap structures.

An MC/MC configuration contains adjacent middle and center pieces rather than completely separated edge pairs. Instead of memorizing a new MC/MC algorithm, the remaining pieces can be solved by applying the known local structures separately:

1. Use an **MM swap** to solve the middle-related target pieces.
2. Use a **CC swap** to solve the center-related target pieces.

This example illustrates an important principle of the minimal-memory approach:

A complicated residual configuration does not necessarily require a new algorithm. It can often be decomposed into familiar local swap operations.

By recognizing the underlying local structures, beginners can solve additional configurations without increasing the number of algorithms they need to memorize.

5. Memory Consideration

The major advantage of this approach is the reduction of algorithmic memory requirements.

A beginner does not need to memorize every possible 6×6 L2C configuration. Instead, the beginner only needs to remember:

- the intuitive reduction strategy;
- how to recognize common local structures;
- the TT, MM, and CC commutator reference cards;
- how transformations and local repairs can simplify unfamiliar configurations.

Because the MM and CC commutators preserve other completed center colors, the beginner can apply them sequentially without repeatedly rebuilding solved portions of the centers.

The reference cards in Figure 8.1 serve as a long-term memory aid. If a beginner forgets an algorithm after not using a 6×6 cube for some time, the visual patterns provide a quick way to recover the required operations.

6. Conclusion

The larger center structure of a 6×6 cube does not necessarily require a proportional increase in memorized algorithms.

By combining intuitive reduction, structural recognition, and commutator-based local swaps, a beginner can solve 6×6 L2C with a significantly reduced memory burden.

The main idea can be summarized as:

Reduce first, recognize the local structure, and solve the remaining pieces using a few reliable operations.

Acknowledgments

I would like to express my sincere appreciation to bcube and the SpeedSolving.com cubing community for their helpful comments, discussions, and generous sharing of knowledge. I am particularly grateful to bcube for introducing me to a more structural way of thinking about center solving through commutators and conjugates, and for sharing the AnimCubeJS simulator. His insights encouraged me to look beyond individual move sequences and better understand the common structures underlying different center configurations.

The Wang Method Series

Notebook #9 (Version 1.0)

Understanding Edge Parity on Odd-Order Rubik's Cubes

A Transition-Based Method for 5×5, 7×7, 9×9, and Larger Odd-Order Cubes

Laung-Terng (LT) Wang

September 9, 2026

1. Introduction

A standard 3×3×3 Rubik's Cube is subject to orientation and permutation constraints that make certain apparent last-layer states impossible. In particular, a single flipped edge cannot occur through legal moves, and two edges cannot be exchanged by themselves while all other pieces remain fixed. These are the kinds of impossible reduced states commonly described as OLL parity and PLL parity on larger cubes.

Most higher-order cubes are solved using the 3×3 reduction method. For an odd-order cube such as the 5×5, 7×7, or 9×9, the solver first builds the six centers and pairs the edge pieces to form twelve complete composite edges.

After reduction is complete, the cube can be treated as an effective 3×3 and solved using only outer-layer moves with a familiar 3×3 method, such as the beginner method or beginner CFOP. The reduced state must satisfy the corresponding 3×3 orientation and permutation constraints. Thus a genuine single-composite-edge-flip or isolated two-composite-edge exchange cannot remain after complete reduction of an odd-order cube.

The parity studied in this notebook occurs earlier, during the pairing of the final composite edge, before reduction is complete. For this reason, we call it edge parity, or more specifically edge-pairing parity, rather than OLL parity or PLL parity.

The familiar long algorithm often called an “OLL parity algorithm” can still be used to repair this edge-pairing condition. However, the name of an algorithm does not determine the mathematical nature of the state it repairs.

This distinction provides the starting point for our study of 5×5, 7×7, 9×9, and general odd-order cubes. As the cube becomes larger, more wing pieces appear on each edge and the number of possible visible patterns grows. The goal of this notebook is to replace memorization of many cases with one systematic transition rule.

Consider a 7×7 edge on which Yellow (Y) is the reference color and Blue (B) is the opposite visible color. At first glance, we might imagine that, after all twelve edges have been paired, one edge could appear as YBBYYYY or YYYYBBY.

These are not legal fully paired edge states. A 7×7 has two different wing orbits. In either proposed pattern, the two B wing positions belong to different wing orbits. A wing piece can be permuted among positions belonging to its own orbit, but it cannot become a wing piece of another orbit.

Therefore, the two B wings cannot be treated as two independently flippable pieces. The better question is: can two wing pieces from different orbits simply be exchanged with each other? The answer is no. The orbit structure prevents that exchange.

This gives a concrete reason why the proposed asymmetric patterns YBBYYYY and YYYYBBY cannot occur as fully paired last-edge states. It also motivates the key viewpoint used in this notebook: apparent wing orientation is constrained by wing permutation within each wing orbit.

Central Rule:

Count each visible-state transition ($0 \leftrightarrow 1$) from the outermost wing toward the midge. Each transition requires one appropriate edge-parity algorithm.

2. Midge, Wings, and Wing Orbits

Before using 0 and 1, we first need three simple terms.

Midge. On an odd-order cube, the piece at the exact middle of an edge is called the midge. There is one midge on each edge.

Wing. The other edge pieces on either side of the midge are called wings.

Wing orbit. Wings that are the same distance from the midge belong to the same wing orbit. An orbit contains all equivalent wing pieces around the cube. On one edge, we see two corresponding positions from each orbit, one on each side of the midge. A wing piece moves among positions belonging to its own orbit; it does not move into a different wing orbit.

5×5: $W_1 \mid M \mid W_1$ (1 wing orbit)

7×7: $W_1 \mid W_2 \mid M \mid W_2 \mid W_1$ (2 wing orbits)

9×9: $W_1 \mid W_2 \mid W_3 \mid M \mid W_3 \mid W_2 \mid W_1$ (3 wing orbits)

2.1 Using 0 and 1 to Describe Orientation

During edge pairing of the last edge, we use the orientation of the midge as a convenient visual reference. The midge can be any color; its actual color does not matter.

0 = same visible orientation state as the midge reference

1 = opposite visible orientation state

Therefore, 0 does not always mean Yellow and 1 does not always mean Blue. If the midge happens to be Yellow in a drawing, Yellow represents 0 only for that example. The binary symbols describe the visible state of a wing orbit; they do not mean that an individual wing can physically flip in place by itself.

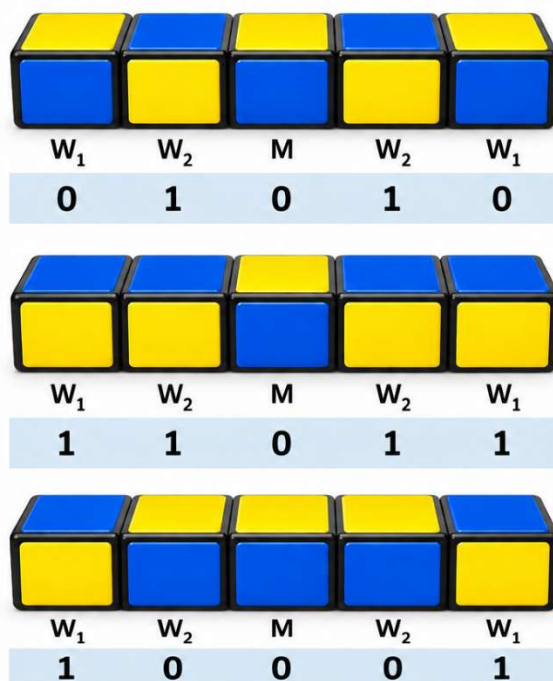
Figure 9.1 illustrates how the midge provides the reference for converting visible colors into binary states. Wings showing the same visible orientation as the midge are assigned 0, while those showing the opposite orientation are assigned 1.

2.2 Orientation as a Visible Effect of Wing Permutation

A crucial point is that a wing does not behave like an independently flippable 3×3 edge piece. Its apparent orientation at an edge location is tied to its permutation within its wing orbit. In a given edge location, such as UF, a wing from a particular orbit can occupy either of the two corresponding positions of that orbit. Moving between those positions changes the visible orientation relationship.

Thus, what looks visually like a wing-orientation problem is more fundamentally constrained by wing permutation. This is why asymmetric patterns such as YBBYYYY and YYYYYBBY cannot simply be treated as additional last-edge parity cases. The binary 0/1 notation used in this notebook remains useful as a visual description, while the underlying physical mechanism is permutation within wing orbits.

Colors to Binary States



In these examples, Yellow (Y) = 0 and Blue (B) = 1, with the Yellow midge on the Up (Top) face as the reference.

Figure 9.1. Examples of converting visible 7×7 last-edge colors into binary orientation states.

3. The 5×5 Base Case

A 5×5 edge has one wing orbit surrounding the midge:

$$W_1 \mid M \mid W_1$$

In the last-edge parity patterns considered here, the two corresponding wing positions from the same orbit share one visible binary state. Therefore, one wing orbit gives only two possibilities: 0 or 1.

$$2^1 = 2$$

Table 9.1. Edge states for the 5×5 cube.

Full edge state	One side to midge	Transitions	Algorithms
000	00	0	0
101	10	1	1

The solved state 000 has no transition. In state 101, one side is 10. Moving toward the midge gives one orientation transition, $1 \rightarrow 0$, so one parity-algorithm application is required. This is the base case for the general deduction.

3.1 Algorithm 1 — 5×5 Edge-Parity Algorithm

For the 5×5 base case, the one-transition state 101 can be repaired with the following 18-move edge-parity algorithm:

$$Rw \ U2 \ x \ Rw \ U2 \ Rw \ U2 \ Rw' \ U2 \ Lw \ U2 \ 3Rw' \ U2 \ Rw \ U2 \ Rw' \ U2 \ Rw'$$

We call this Algorithm 1. It provides the 5×5 base algorithm before we introduce the two 7×7 layer-width variants.

4. The 7×7 Cube: Four Possible Patterns

A 7×7 edge has two wing orbits: an outer-wing orbit and an inner-wing orbit.

$$W_1 \mid W_2 \mid M \mid W_2 \mid W_1$$

For each orbit, the two corresponding wing positions are linked by the permutation structure of that wing orbit. In the last-edge parity patterns considered here, each orbit therefore contributes one independent visible binary state: 0 or 1. With two wing orbits, the 7×7 has

$$2^2 = 4$$

symmetric combinations, including the solved state: 00000, 10001, 01010, and 11011. An asymmetric string such as 00011 is not one of these cases because it assigns different visible states to corresponding positions of the same wing orbit. This is not merely a visual symmetry assumption; it reflects the underlying wing-orbit permutation constraint.

Table 9.2. Edge states for the 7×7 cube.

Full edge state	One side to midge	Transitions	Algorithms
00000	000	0	0
10001	100	1	1
01010	010	2	2
11011	110	1	1

4.1 The Two 7×7 Edge-Parity Algorithm Variants

For the 7×7 cube, there are two possible transition boundaries, so two layer-width versions of the edge-parity algorithm are needed. Continuing the numbering after the 5×5 base case, we call them Algorithm 2 and Algorithm 3.

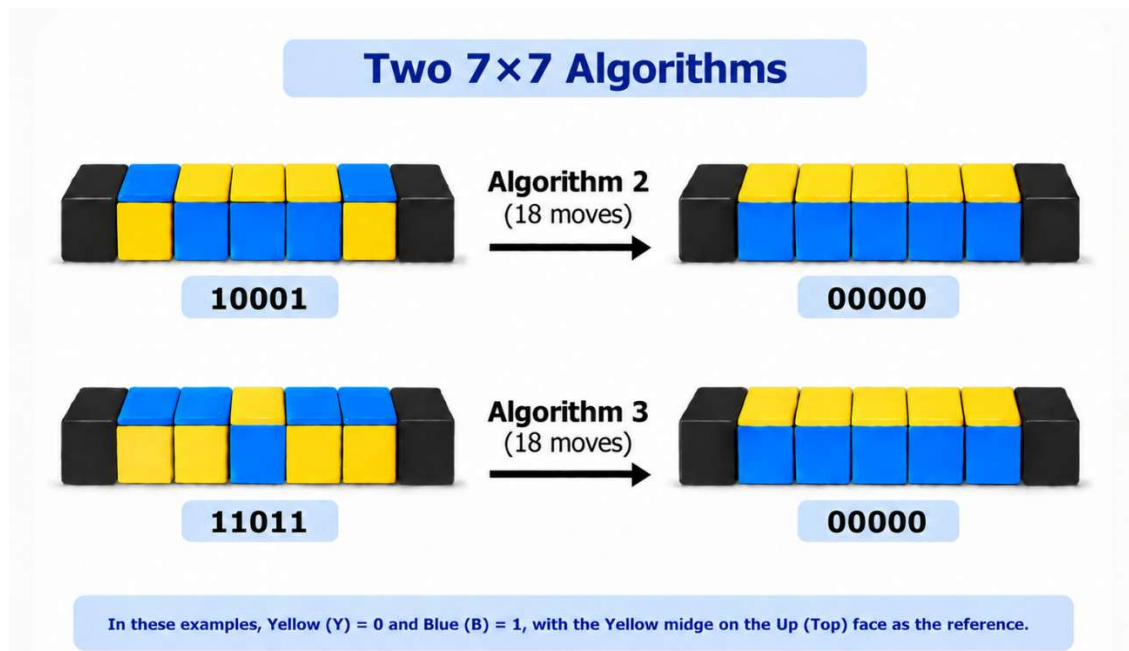


Figure 9.2. The two 18-move edge-parity algorithms needed for the two one-transition 7×7 cases.

Algorithm 2 — solves 10001 → 00000 (18 moves)

$$Rw \ U2 \ x \ Rw \ U2 \ Rw \ U2 \ Rw' \ U2 \ Lw \ U2 \ 5Rw' \ U2 \ Rw \ U2 \ Rw' \ U2 \ Rw'$$

Algorithm 3 — solves 11011 → 00000 (18 moves)

$$3Rw \ U2 \ x \ 3Rw \ U2 \ 3Rw \ U2 \ 3Rw' \ U2 \ 3Lw \ U2 \ 4Rw' \ U2 \ 3Rw \ U2 \ 3Rw' \ U2 \ 3Rw'$$

Figure 9.2 illustrates the two one-transition cases on the 7×7 cube. Algorithm 2 repairs the outer transition in 10001, while Algorithm 3 repairs the inner transition in 11011, bringing either pattern to the solved state 00000.

4.2 The Two-Transition Case: 01010

The state 01010 is especially useful because one side together with the midge is 010. It contains two orientation transitions:

$$0 \rightarrow 1 \rightarrow 0$$

Therefore, two algorithm applications are needed. For the 7×7, Algorithm 2 is applied first, producing the intermediate state 11011. Algorithm 3 then solves that state.

$$01010 \rightarrow 11011 \rightarrow 00000$$

Figure 9.3 shows how the two algorithms work together when both transition boundaries are present. Algorithm 2 first changes 01010 to 11011 by repairing the outer transition; Algorithm 3 then repairs the remaining inner transition, producing the solved state 00000.

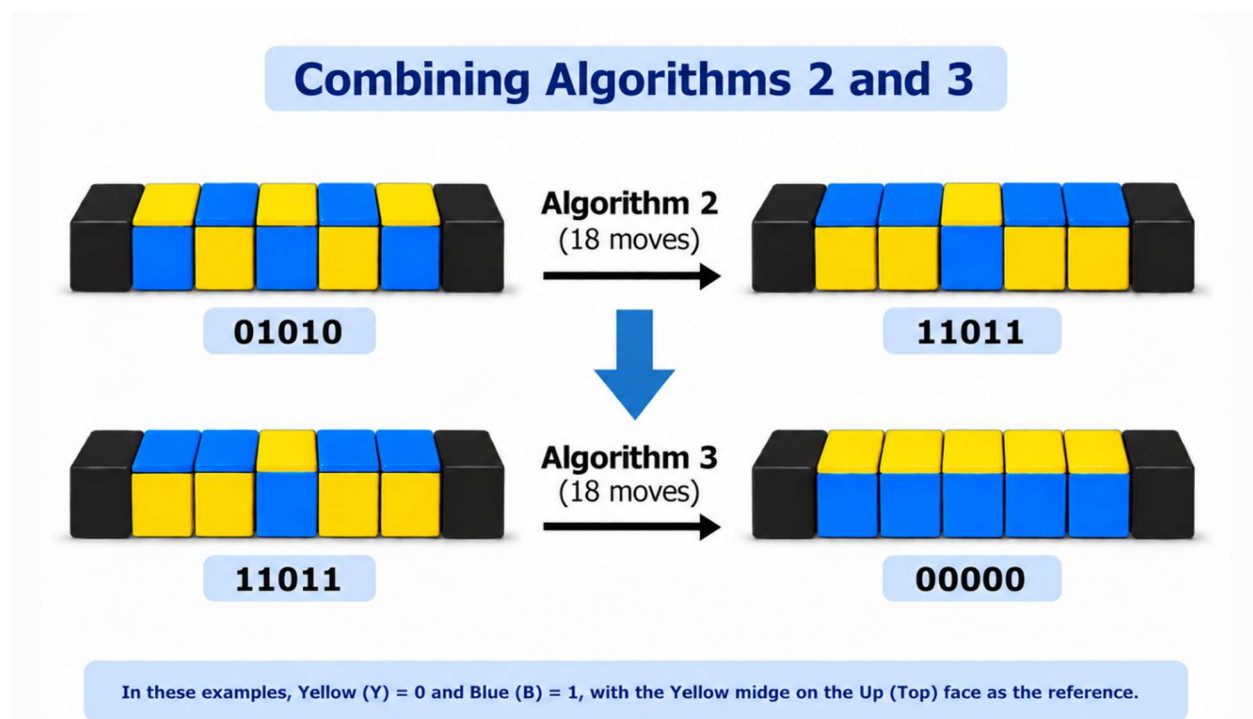


Figure 9.3. The 7×7 case 01010 requires two transitions and is solved by Algorithm 2 followed by Algorithm 3.

5. Extension to the 9×9 Cube

A 9×9 edge has three wing orbits surrounding the midge. Therefore, there are

$$2^3 = 8$$

symmetric binary states, including the solved state. There are also three possible transition boundaries, so a 9×9 needs three corresponding layer-width versions of the edge-parity algorithm. Their exact layer widths depend on which of the three transition boundaries is being repaired.

Table 9.3. Edge states for the 9×9 cube.

Full edge state	One side to midge	Transitions	Algorithm applications
0000000	0000	0	0
1000001	1000	1	1
0100010	0100	2	2
0010100	0010	2	2
1100011	1100	1	1
1010101	1010	3	3
0110110	0110	2	2
1110111	1110	1	1

The table shows why the number of 1s is not the answer. For example, 1110 has only one transition, while 1010 has three transitions. The required algorithms are determined by the transition boundaries, not simply by how many wing orbits are in state 1.

6. The Transition Rule

For any odd-order cube, write the edge as a symmetric binary sequence around the midge. Because corresponding positions from each wing orbit have the same state, only one side together with the midge needs to be inspected.

Number of required algorithm applications = number of adjacent visible-state transitions from the outermost wing to the midge.

Equivalently, the transitions may be counted from the midge outward; the number of transitions is the same in either direction.

A transition occurs whenever two neighboring visible states differ, $0 \leftrightarrow 1$. Each possible transition boundary has a corresponding layer-width version of the parity algorithm. Apply the appropriate version for each transition boundary until the last edge is paired.

7. Why the Rule Works for Every Odd-Order Cube

We do not need to memorize or experimentally list every case for 11×11, 13×13, and larger odd-order cubes. The rule follows by induction.

7.1 Base Case: 5×5

The 5×5 has one wing orbit. Its two states, 000 and 101, require zero and one algorithm application. Thus the transition rule holds for the base case.

7.2 Inductive Step

Increasing an odd-order cube from $N \times N$ to $(N+2) \times (N+2)$ adds exactly one new outer wing orbit. This adds one new binary state to the one-sided representation and therefore at most one additional transition.

$$N \times N \rightarrow (N + 2) \times (N + 2)$$

Assume the transition rule is correct for an odd $N \times N$ cube. When the new outer wing orbit is added, all of the old inner states and their transitions stay the same. There is only one new comparison: the new outer state versus the state that was previously outermost.

If the two states are the same, no new transition is created. If they are different, exactly one new transition is created. Therefore, moving from $N \times N$ to $(N+2) \times (N+2)$ adds either zero or one required algorithm application. This is the inductive step.

Since the rule is true for 5×5 and remains true each time one new wing orbit is added, it applies to all odd-order cubes of size 5×5 and larger.

7.3 General Mathematical Form

For an odd $N \times N$ cube, the number of wing orbits is

$$m = \frac{N - 3}{2}$$

Each wing orbit has two possible orientation states, 0 or 1. Therefore, the number of symmetric binary edge states is

$$2^m = 2^{\frac{N - 3}{2}}$$

Let $b_1, b_2, \dots, b_m$ be the one-sided wing-orbit states from the outermost wing toward the midge, and let the midge be $b_{m+1} = 0$. Then the number A of required algorithm applications is the number of adjacent state changes:

$$A = \sum_{i=1}^m |b_i - b_{i+1}|$$

The maximum occurs when the states alternate at every possible boundary:

$$A_{\max} = \frac{N - 3}{2}$$

8. A Beginner's Solving Procedure

1. At the last edge during edge pairing, use the midge orientation as the visual reference state 0.
 2. Write each wing orbit as 0 if it shows the same visible orientation state as the midge reference, or 1 if it shows the opposite visible state.
 3. Because the pattern is symmetric, inspect only one side from the outermost wing to the midge.
 4. Count every $0 \leftrightarrow 1$ transition.
 5. For each transition boundary, apply the corresponding layer-width version of the edge-parity algorithm.
 6. Continue until all states are 0 and the last edge is paired.
-

9. Why the Method Is Useful

The transition method replaces a rapidly growing list of parity cases with one visual rule. A 5×5 has 2 symmetric states, a 7×7 has 4, a 9×9 has 8, and an 11×11 has 16. In general, with m wing orbits there are

$$2^m$$

possible symmetric binary states. These cases do not need to be memorized one by one. The solver only needs to identify the transition boundaries and use the corresponding layer-width algorithms.

The method is intended to be understandable at two levels. A young beginner can use the pictures and simply count the changes. An older student can continue to the induction and see why the same rule works for every odd-order cube.

10. Concluding Remarks

After complete reduction, odd-order cubes do not leave a genuine single-edge-flip or isolated two-edge-exchange state in the effective 3×3 stage. The parity discussed here belongs to the last-edge stage of edge pairing. The binary notation provides a simple visual language, while the underlying behavior is constrained by wing permutation within wing orbits and by the midge reference.

The 5×5 provides the base case. Going from $N \times N$ to $(N+2) \times (N+2)$ adds one new outer wing orbit, one new binary state, and at most one new transition. This gives a natural inductive proof for all odd-order cubes.

Count the transitions. Use the algorithm for each transition boundary.

Acknowledgments

The author gratefully acknowledges Lee Dearn (stoic) of the SpeedSolving.com community for his helpful explanations concerning edge parity on odd-order cubes, particularly the distinction between edge parity during edge pairing and OLL/PLL parity after reduction, and the behavior of corresponding wing pieces. The author also thanks SpeedSolving.com member bcube for emphasizing the wing-orbit and permutation viewpoint, especially the insight that apparent wing orientation is determined by permutation within a wing orbit. Their comments helped motivate the transition-based interpretation and the clearer distinction between visible binary states and the underlying physical behavior developed in this notebook.